Novel algorithms for simulation and optimisation of periodic processes operating at cyclic steady state

Susana Teixeira Luis Fialho Medinas

Thesis to obtain the Master of Science Degree in

Chemical Engineering

Supervisors: Prof. Dr. Henrique Aníbal Santos de Matos
Dr. Klaas Martÿn Nauta

Examination Committee

Chairperson: Prof. Dr. Carlos Henriques
Supervisor: Prof. Dr. Henrique Aníbal Santos de Matos
Members of the Committee: Prof. Dr. Pedro Castro

2016

This page was intentionally left blank.

Acknowledgments

First of all I would like to thank Professor Henrique Matos for all his support and for the motivational and kind words given to me along this journey. I also would like to express my gratitude to Professor Costas Pantelides for the opportunity to work at Process Systems Enterprise Ltd, and turning the project of this thesis into an amazing experience.

I give my special thanks to Maarten, my supervisor at PSE, without whom I would never have been able to accomplish this thesis. All his knowledge in several subjects were essential during this past few months. I also want to thank all the people at PSE that helped me, always with kindness and availability, never saying "no" whenever I needed most.

Last but not least I appreciate all my friends' help either in Portugal or back in London and of course a huge thank you for my beloved family which no matter the distance were always there for me.

T*his page was intentionally left blank.*

Abstract

The study of novel algorithms for simulation and optimisation of periodic processes operating at cyclic steady state is of major importance. This state is equivalent to the steady state of a periodic process.

Pressure swing adsorption (PSA) is a periodic adsorption process which tends to a cyclic steady state. It is a Process in which the pressure swings (varies) between two values along each cycle. The higher pressure promotes the adsorption step while the lower allows the regeneration of the adsorbent by desorption. This technique is used as a separation process in the industry. The stream passing through the column gets enriched in the less adsorbed components while the more adsorbed components are removed from this stream by the adsorption. This process is often called Rapid Pressure Swing Adsorption (RPSA) when the steps of the cycle are very fast, or the dimension of the adsorption bed(s) is very small.

In this work, an already existent example of RPSA for production of enriched oxygen air in gPROMS® ModelBuilder was converted into gPROMS® ProcessBuilder and validated displaying a small deviation in the two Key Performance Indcators monitored (Purity and Recovery).

Wegstein method was applied to models with simple functions in gPROMS® ProcessBuilder and to the one bed RPSA example gPROMS® ProcessBuilder that was developed during this thesis project. In order to do this, a program with an algorithm was developed in Python. This program ensures the communication between the two software, gPROMS® and Python, in the correct points of the program's execution. Wegstein acceleration method (developed in Python language) is integrated in the program in such a way that the acceleration of the convergence is applied to the process variables specified by the user. The accelerated solution is returned to the user or even to gPROMS® itself (in case the user sets, in the program, the number of Wegstein method accelerations higher than 1 - which is very helpful since for this type of processes one acceleration is very probably not enough for convergence). Wegstein method shows better convergence when compared to successive substitution.

A Jacobian-free Newton Krylov (JFNK) method was applied to the one bed RPSA gPROMS® ModelBuilder example with the help of a cyclic solver developed at PSE. JFNK method shows a much faster convergence when compared to successive substitution. A sensitivity analysis and parametric study was then executed with the RPSA ModelBuilder example, showing that the preconditioning has a huge impact on the performance of the solver.

Keywords

gPROMS, Periodic Processes, Pressure Swing Adsorption, Cyclic-steady state, Wegstein method, Jacobian-Free Newton-Krylov method, Python

This page was intentionally left blank.

Resumo

O estudo de novos algoritmos para a simulação e optimização de processos periódicos no estado cíclico estacionário é de extrema importância. Este estado é equivalente ao estado estacionário para processos periódicos.

Pressure swing adsorption (PSA), Adsorção com oscilação de Pressão, é um processo de adsorção periódico que tende para um estado cíclico estacionário. É um processo no qual a Pressão oscila entre dois valores ao longo de cada ciclo. A Pressão mais elevada promove o passo da adsorção enquanto que a mais inferior permite a regeneração do adsorvente através da desorção. Esta técnica é um dos processos de separação mais usados na indústria. A corrente que passa através da coluna enriquece o seu conteúdo nos componentes menos adsorvidos, ao passo que os components mais adsorvidos são removidos desta corrente pela adsorção. Este processo é muitas vezes nomeado de Rapid Pressure Swing Adsorption (RPSA), Adsorção com Oscilação de Pressão Rápida, quando os passos que constituem cada ciclo são muito curtos, ou a dimensão da(s) coluna(s) de adsorção é muito pequena.

Na presente tese, um exemplo já existente de RPSA para produção de ar enriquecido em oxigénio em gPROMS® ModelBuilder é convertido em gPROMS® ProcessBuilder e validado demonstrando um pequeno desvio nos dois Key Performance Indicators monotorizados (Pureza e Recuperação).

O método de Wesgtein é aplicado a modelos com funções em gPROMS® ProcessBuilder e ao exemplo de RPSA de gPROMS® ProcessBuilder que foi desenvolvido ao longo deste projeto. De modo a aplicar este método, um programa com um algoritmo foi desenvolvido em Python. O programa garante a comunicação entre os dois *software*, gPROMS® e Python, nos pontos correctos, durante a execução do programa. O método de aceleração de Wegstein (desenvolvido em Python) é integrado no programa de forma a que a aceleração da convergência é aplicada às variáveis do processo especificadas pelo utilizador. A solução acelerada é devolvida ao utilizador ou ao próprio gPROMS® (no caso do utilizador definir, no programa, que o número de acelerações deste método é superior a 1 – o que é bastante vantajoso, visto que, para este tipo de processos, uma aceleração é muito provavelmente insuficiente para a convergência). O método de Wegstein mostrou melhores resultados de convergência quando comparado com a Substituição Sucessiva.

Um método *Jacobian-free Newton Krylov (JFNK),* isento de cálculo de um Jacobiano, baseado no método de Newton, é aplicado ao exemplo de RPSA no model Builder através de um solver para processos cíclicos desenvolvido na PSE. JFNK é um metodo que apresenta uma convergência bastante mais rápida em relação à substituição sucessiva. Uma análise de sensibilidade e estudo paramétrico é então realizado para o exemplo de RPSA no gPROMS® ModelBuilder, mostrando que o pré-condicionamento do Sistema tem um impacto muito relevante na performance do *solver*.

Palavras-Chave

gPROMS, Processos Periódicos, Adsorção com oscilação de Pressão, Estado cíclico estacionário, Método de Wegstein, Método Jacobian-Free Newton-Krylov, Python

This page was intentionally left blank.

Contents

List of Figures

This page was intentionally left blank.

List of tables

This page was intentionally left blank.

Notation and glossary

Symbol	Description	unit
A	Bed area	m^2
C	Concentration	mol m^{-3}
db	Bed diameter	m
dp	Adsorbent particle diameter	m
D_l	Dispersion coefficient of component i	m^2/s
Ki	Mass Transfer coefficient of component i	s^{-1}
$\boldsymbol{m_i}$	Coefficient of Langmuir isotherm	mol/(kg.Pa)
MW	Molecular weight	kg/mol
Mfeed; Mwaste; Mproduct	Amount of material fed, escaping to/from the column; Amount of product	mol m^{-2} s^{-1}
P	Bed pressure	Pa
q	Concentration of gas phase components adsorbed on solid phase	mol /kgsolid
qeq	Equilibrium concentration of gas phase components adsorbed on solid phase	mol/kgsolid
T	Temperature of both fluid and solid phase	K
u	Fluid superficial velocity	m/s
v	Fluid dynamic viscosity	Pa. s

x	Component mass fraction	kg kg^{-1}

Greek Letters

Symbol	Description	unit
$\boldsymbol{\varepsilon_{bed}}$	Bed void fraction	m^3_{void}/m^3_{bed}
$\boldsymbol{\varepsilon_p}$	Particle void fraction	m^3_{void}/m^3_{bed}
$\boldsymbol{\varepsilon_{tot}}$	Total void fraction	m^3_{void}/m^3_{bed}
$\boldsymbol{\rho}$	Bed fluid density	kg_{fluid}/m^3_{bed}
$\boldsymbol{\rho_{bed}}$	Bed density	$kg_{adsorbent}/m^3_{bed}$
$\boldsymbol{\rho_w}$	Wall density	kg/m^3
$\boldsymbol{\tau}$	Tortuosity factor	

i = Component

1. Introduction

Along the years, many separation processes have been employed for fluid separation such as extraction and distillation. However this separation can also be managed with adsorption techniques (which are normally less expensive than other separation processes) and so consequently some Periodical Adsorption Processes (PAP) are preferred to others. In the last years, the costs associated with the energy required for distillation served as a driving force to improve research on adsorption (Ruthven, 1984).

Generic periodical adsorption processes (PAP) are classically employed for separation processes, such as gas purification, gas drying and solvent vapour recovery applications and bulk gas separation applications (Oliver J. Smith, 1992). Therefore, improving modelling and optimisation techniques of these type of processes is very important. Nevertheless, these types of processes have very specific characteristics requiring a special treatment.

When a system is in its initial state and different inputs are given to the system, these new inputs disturb the system and deviate it from its steady and initial state to a transient state. In a transient state, the variables of the system vary over time, and sometimes, after a constant behavior in the input, the system reaches a new steady state. However, looking at periodical processes instead of considering a steady behavior of the system, we need to look for a constant periodical behavior repeated over each cycle.

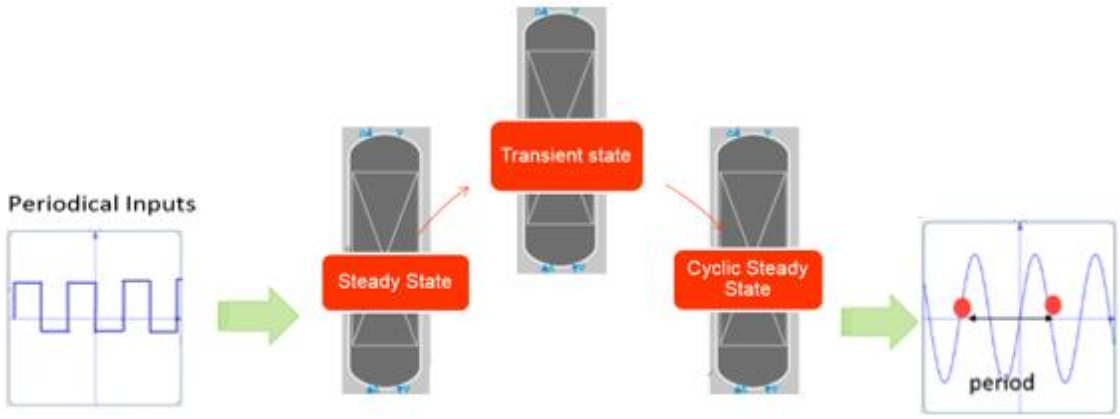

Figure 1 – Sates of a system disturbed by a Periodic Input

Periodical processes such as cyclic moving bed processes and cyclic fixed bed processes (like Pressure Swing Adsorption), are processes whose state can be modelled by a periodic function. This type of chemical periodic processes are operated cyclically and when subjected to a periodic input signal for a certain amount of time, they converge to a cyclic-steady-state (CSS). CSS is reached when the state values at the start of cycle k are equal to the ones at the start of cycle k+1. This is a quite interesting condition in the operation, therefore the convergence to this state needs to be well accomplished, namely by a good fitted models.

1.1 Relevance and motivation

Modelling and simulation of PAPs needs to address the intrinsic complexity of these processes, which manly arises from their periodic nature. While studying these processes reach the cyclic-steady-state (CSS).is imperative. However, the convergence to CSS is highly time consuming (can take several hours). For instance, considering direct substitution, also known as successive substitution, which is using the final value of an iteration as the initial guess for the next iteration, seeing that most of these processes need many previous cycles until reaching this state, the convergence to the CSS can be a huge drawback in the process.

This successive substitution approach can be prohibitive for various applications such as optimization, which requires a large number of iterations for each one of the input conditions and process parameters in study. Since each convergence takes several hours, the entire optimization can take weeks. A more efficient way to deal with this issue is required and therefore was appointed as the motivation to this work.

Therefore, the process may be accelerated in many different ways considering that the simulation of a single cycle is a nonlinear function mapping from the initial state to the final state (system state at end of the cycle is a function of system state at start of the new cycle). A new approach can be considered by this function using a sequence of cycles as an iteration which converges to the CSS, and then, regarding the results of previous iterations, the state at the start of each iteration can be estimated accelerating convergence.

1.2. Original Contributions

The goals of this project were to investigate algorithms to accelerate simulation and optimisation of periodic processes and integrate these algorithms in PSE's gPROMS® ProcessBuilder product in such a way that they can be used by customers.

A new code to implement the Wegstein method was developed in Python framework with the search on the communication with this module with gPROMS® family.

Consequently several gPROMS® ProcessBuilder models containing simple equations and a RPSA example were simulated with the acceleration approach created in Phython.

Afterwards, a Rapid Pressure Swing Adsorption (RPSA) containing only one adsorption bed to purify oxygen from atmospheric air was converted into gPROMS® model library (gML) in order to be used during this project. The simulation results obtain in the gPROMS® ProcessBuilder using the gML module was compared and validated with the gPROMS® ModelBuilder previously created at PSE.

Finally a prototype cyclic solver, recently created at PSE, was used to simulate the RPSA gPROMS® ModelBuilder until the CSS. The influence of certain solving parameters specified in the performance and convergence of the solver were also studied.

1.3. Dissertation Outline

This thesis is outlined as follows:

Chapter two presents a literature review for this thesis, regarding a brief introduction of the main concepts of Adsorption and Pressure Swing Adsorption process. It also introduces a Mathematical formulation of periodical processes and CSS as well as convergence acceleration approaches and algorithms applied to the processes in study. Finally, some information on python software language is also presented.

Chapter three describes the architecture and results of the *Python* program to accelerate the convergence of simple functions and one simple RPSA.

Chapter four describes the conversion of the RPSA example previously created in gPROMS® ModelBuilder into gML module. Moreover the main equations of the mathematical model, the model configuration and results of these two RPSA examples are also presented. The Cyclic solver configuration is also offered followed by the results of its application.

In chapter five the conclusions of this thesis are presented.

This page was intentionally left blank.

2. Literature Review

2.1 Adsorption and Isotherms

Adsorption is a phenomenon that allows separation of species in a fluid mixture. In adsorption, a molecule from a fluid phase experiences a spontaneous attraction close to the surface of a solid (adsorbent) which causes the molecules to distribute themselves between two phases (Richardson, et al., 2002). To keep the molecular density in the surroundings of the surface greater than in the free-gas phase, a concentration of molecules tends to occur in the solid region.

The phenomenon of physical adsorption will happen if the interactions between gas and solid are weak, involving only van der Waals force. The nature of the surface affects the type of interaction and the way that different molecules will interact with the surface. Selectivity of the process is defined by the substance for the adsorption separation process. Chemisorption processes often have small capacities, which make this a less feasible adsorption process. Normally, separation processes including PSA rely on physical adsorption.

This process can be ruled by either differences in the adsorption equilibrium or adsorption rates (Ruthven, 1984) and results in a reduction of the potential energy. What occurs in the former is that an equilibrium state is achieved some time after the adsorbent contacts with a fluid phase. A thermodynamic limit is set by this condition, which is function of the fluid phase Composition, Temperature, and Pressure. In order to design and model adsorption processes is crucial to have information about the adsorption equilibrium of the different species (Serbezov, 2001) (Equilibrium Theory analysis of dual reflux PSA for separation of a binary mixture, 2004). Adsorbents have a large surface area per unit mass, being porous solids. They allow a separation of lots of different species since the molecules have specific interactions with their surface. (Polanyi, 1932) (Ruthven, 1984) (Yang, 2003) (Staudt, 2005).

Adsorption dynamics and breakthrough behaviour of a bed depends on the layers of the bed and on the mixture. It is very important to study these behaviours since the adsorption step is probably the most important step for optimization of the procedure. In order to validate used models for the simulation of this process, the breakthrough curves of simulation and experimental work need to be compared. In fact the amount adsorbed and the role of the species and the interval required by each step (adsorption and desorption) can be obtain analysing the breakthrough curve (Bastos-Neto, 2011). Of course the adsorbent material and the knowledge of the thermodynamics and the kinetics of a given adsorbate or adsorbent system is crucial for this and can be achieved through the equilibrium adsorption isotherms and breakthrough experiments, respectively (Gittleman C. et al, 2005)

Adsorption is a very complex mechanism, so several models have been developed to describe it. Isotherms are curves that can relate, in equilibrium, the amount of a substance from a fluid phase held in a solid-phase. Isotherms can vary substantially, and their shape is crucial while choosing the adsorbent that will be used. There are linear, favourable and unfavourable isotherms. A favourable isotherm shows a convex upward curvature, and the dimensionless adsorbed phase is always higher than the dimensionless fluid phase concentration (Conney, 1998). An isotherm that is favourable leads to sharp fronts in the adsorption step and describes an unfavourable

desorption. Meanwhile, a concave isotherm results in broadening fronts. In fact, when the breakthrough concentration of solute is reached in the effluent, a high degree of the adsorption capacity of the bed will have been used, and the column needs to be 'fixed'. These fronts spread proportionally to travel distance and time. Different isotherms apply to specific cases. For instance for low concentrations, the Langmuir isotherm is described by Henry's law and shows a saturation limit at high concentrations. Isotherms can be combined, for instance, Langmuir isotherm can be combined with the Freundlich isotherm.

2.2 Adsorbent

The main task of defining an adsorption unit is selecting the adsorbent to be employed correctly, since the behaviour of the PSA unit is mainly determined by the adsorbent employed for the separation (Knaebel, 2004). Defining an effective strategy to regenerate the adsorbent is also a very important task, and there is where engineering takes place.

The adsorbent is usually packed in fixed beds, and it can be shaped into various forms, such as spherical pellets or extruded. Alternatively, the pressure drop in the system with honeycomb monolithic structures can be significantly lower than a packed bed system (Gadkaree, 1998). To develop this technology studies in material science and engineering are important in the discovery of new adsorbents and new and more efficient ways to use and regenerate them.

To be able to size the adsorbent, since adsorption can vary by many orders of magnitude, the equilibria information of adsorption isotherm is clearly the first hand information needed. This information can be obtained experimentally from very low pressure (where adsorption affinity can be estimated) to very high pressure to calculate the saturation. With appropriate theories or computer simulation and advances of high speed computer and modern tools it is possible to deal with fluids in confined spaces like micropores. Of course reliable experimental data is required for validation of this theories and simulations. (Lee, 2003). Another fact that may be important is the time required to achieve the equilibrium state, especially when the size of the pores of the adsorbent are similar to the size of the molecules to be separated).

A lot of variety is available nowadays, being part of the most common adsorbents, activated aluminas and silica gels are both useful desiccants, however silica gel shows high capacities at low temperatures and activated alumina shows the same for high temperatures. Activated carbons (ACs) are very competitive due to a lower cost, high surface area and amenability to pore structure modification, and surface functionalization (García S. et al, 2013). Zeolites, being porous crystalline aluminosilicates, show a specific characteristic when compared to other adsorbents, they have a structure of a combination of SiO_4 and AlO_4, connected in several regular arrangements through shared oxygen atoms, forming an open crystal lattice which determines the uniform micropore structure, with no distribution of pore size. (Ruthven, 1984)

The bed can also be layered so that adsorbents with different adsorption properties can be used to combine and improve the process performance by utilizing the adsorbents' inherent potentials (Y. Lu, et al., 2004). Due to the

differences between the feed components, multilayer beds were introduced in the PSA process, allowing a high purity product requiring a more compact process, a low number of columns and valves, and a reduction overall product cost (this is more frequent when the feed gas contains multiple components).

When the number of gases to be removed from the feed is considerable, different layers of adsorbents can be used to remove each particular group of contaminants, or contaminants. The first layer of is commonly silica or alumina to retain moisture from the feed. Carbon dioxide and hydrocarbons such as methane can be adsorbed selectively by an activated carbon layer. These can be adsorbed by zeolite, but in this type of adsorbent these species start to accumulate throughout the cycles, since they cannot be readily desorbed by decreasing the pressure. Therefore, it is necessary to prevent these components to reach the layer of adsorbent in which they adsorbs strongly, so that desorption is achievable not requiring vacuum. Therefore, a common constraint to designing a layered PSA unit is that, CO_2 and water vapour must not reach the zeolite layer (Chlendi, et al., 1995). Also the lighter impurities (CH_4, CO_2 and N_2) cannot break through the entire bed. Zeolite purpose is to remove carbon monoxide, nitrogen, argon and residual methane (Jee, et al., 2001)

In conclusion, there are several particular factors that lead to selecting a specific adsorbent for a particular process. Among others it could be highlighted the multicomponent adsorption equilibrium capacities, selectivity's, and available surface area, (Sircar, et al., 2000)

2.3 Periodical Adsorption Processes

Periodical Adsorption Processes (PAPs) are used to separate at least one component from a mixed stream, requiring a least an adsorbent that preferentially adsorbs that one component. They are characterized by two main steps Adsorption and Desorption. In the Adsorption step, the preferentially adsorbed species are gathered from the feed, while in the desorption step, these species are removed from the adsorbent. This step is also called regeneration (Ruthven, et al., 1994)

There are different types of PAPs depending on the technique used to regenerate the solvent. The desorption can be managed by changing process parameters such as Temperature and Pressure, since that affects the adsorption equilibrium. In the Pressure Swing Adsorption (PSA) process this occurs due to the reduction of the partial pressure of the gas phase. PSA cycles are usually shorter but require more adsorbent per unit time. In a Vacuum Swing Adsorption (VSA) process, the regeneration step requires the use of vacuum, the main advantage is the fact that the working capacity of the adsorbent is higher under vacuum operation. Adsorption is an exothermic process, meaning the increase of temperature will favour the desorption, so in Temperature Swing Adsorption (TSA), the increase in temperature regenerates the adsorbent (Grande, 2012). This type of process is useful when the concentration is low and the adsorption step takes long (Liu, 2011). Composition Swing Adsorption (CSA) is assured by a change in the composition of the fluid phase. There is also Rapid Pressure Swing Adsorption (RPSA), which is a PSA with a small length steps.

Pressure swing adsorption is preferred to other processes when the concentration of the components to be removed is high. In this case, loading the column with the heavy component is accomplished quite fast and since the pressure of the system can be changed rapidly, the time between adsorption and regeneration is balanced. When the concentration is low, the adsorption step may take much longer and other options like temperature swing adsorption (TSA) can be considered.

Studies of Periodic Processes modelling and optimisation (such as PSA) have been reported in the literature. (Nilchan, et al., 1998) (Jiang, et al., 2003) (Cruz, et al., 2003)

2.4 Pressure Swing Adsorption

Porous solids are known to reversibly adsorb large volumes of vapour since the eighteen century, however the first application ranges in recovery of aromatic hydrocarbons were in early 1950's. In fact, the concept of reducing the total pressure of the system, so that the adsorbent can be regenerated has been patented in 1932. (Heinrich, 1953)

Only one of the two streams can generally be obtained with the desired purity in PSA. A vast majority of PSA systems are "Stripping-type Pressure Swing Adsorption" (S-PSA) resulting in a pure light component, whereas "Rectifying-type Pressure Swing Adsorption" (R-PSA) allows the collection of a purified heavy component. However the Dual Reflux Pressure Swing Adsorption (DR-PSA) process allows enrichments in the light and heavy components. (Rota, 2015). The stripping type are based on (Skarstrom, 1959) cycle and the rectifying type were developed taking the (New PSA process with intermediate feed inlet position operated with dual refluxes - application to carbon-dioxide removal and enrichment, 1994) and (Equilibrium theory analysis of rectifying PSA for heavy component production, 2002)) . Stripping PSA processes are capable of producing only the light product at high purity from a binary feed gas mixture, since the purity of the heavy product is confined by thermodynamic constraints (Equilibrium theory for solvent vapor recovery by pressure swing adsorption: analytical solution for process performance, 1997). There are various modifications to the typical cycle. PSA is a technique with lots of possibilities where some of them may have impact on energy requirements and even on the efficiency of the unit.

More varieties were introduced in early 1960's but only in 1970's there was an increase in scale and range of this processes. The adsorption in commercial scale requires availability of adsorbent in tonnage quantities with economic cost and this was only managed some years after the first PSA applications.

2.5 Operation of PSA cycles

2.5.1 Skarstrom cycle

Pressure Swing Adsorption systems were the first technology created using the Skarstrom's cycle. In the PSA process the total pressure of the system "swings" between high pressure in feed and low pressure in regeneration.

The stream rich in the less adsorbed species is usually named as raffinate stream. While the stream rich in the heavy components is an extract stream. (Cruz, et al., 2003)

The Skarstrom process is the basis of actual processes. This process includes two adsorbing beds and involves four steps: Pressurization, Adsorption (Feed), Countercurrent Blowdown and Countercurrent Purge. The Skarstrom Cycle, is presented by Skarstrom in 1957 as a process to dry air or other gaseous materials. It is the most well-known pressure-swing-adsorption process and consists in a two-bed , four-step process that does not need an external heat to regenerate the adsorbent, which reduces the requirement for adsorbent material and provides equipment that allows the production of effluent streams rich, in at least, one component. When the feed stream enters the column what happens is that the less adsorbed (light) component passes through the column faster than the other(s). The compounds that are more adsorbed (heavy) will stay in the adsorbent. Therefore, the feed should be stopped before the adsorbent capacity has been exceeded, so that it can be regenerated (by desorbing) avoiding the heavy compounds to travel through the column.

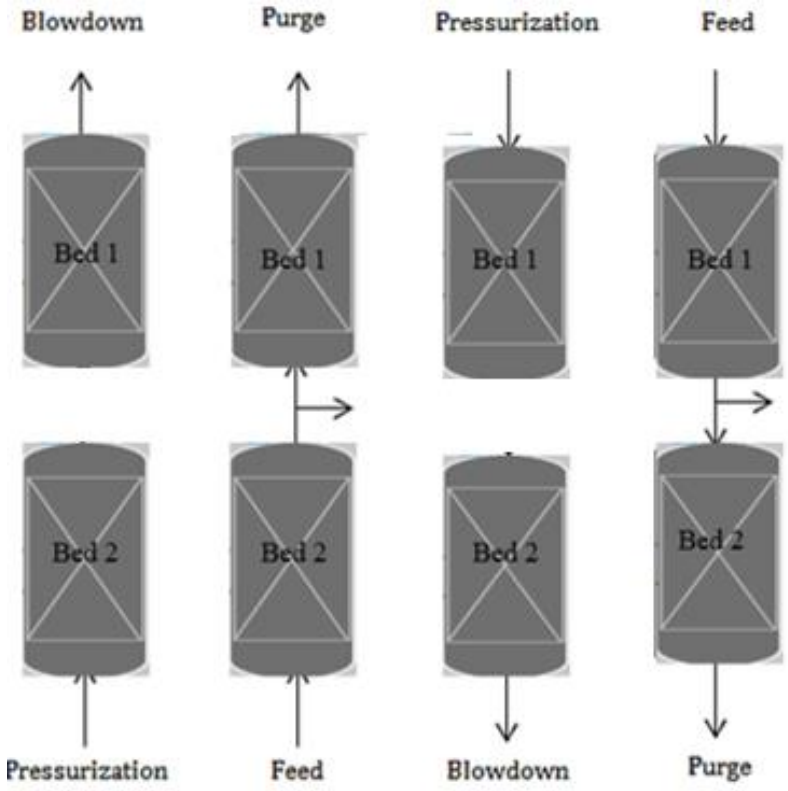

Figure 2 - Sequence of steps in the basic Skarstrom PSA cycle

Figure 2 shows the Skarstrom cycle steps: Pressurization, Feed, Blowdown and Purge of Bed 2, which corresponds to Blowdown, Purge, Pressurization and Feed, for Bed 1. Since the adsorption and desorption are favoured by a higher and lower pressure respectively, firstly, gas is fed to the column ensuring pressurization. The steps are described for bed 2, (at this time, Bed 1 is at the Blowdown step and the valve at the product end of bed 2 is closed, not producing, so the entrance of gas leads to an increase of pressure in Bed 2) and then the actual feed step happens, the valve at the product-end is opened and the bed is producing. After the effluent (non-adsorbed)

product is collected, the product can be used to purge the column, feeding it to Bed 2 at its product end. The flow of the purified component in the reverse direction, with both valves opened, causes the strongly adsorbed components to be flushed back, to the feed end, regenerating the adsorbent. However, before purging, the valve at the product end is quickly opened, causing an abrupt decrease of the pressure and desorption of the not so strongly adsorbed species.

2.5.2 Innovations to the Skarstrom cycle

Since 1957, several innovations have been applied in this process, including new elementary steps such as co-current depressurization elementary step (to an intermediate pressure). This step allows an increase in the concentration of the heavy components, while the outlet stream consists in pure light product. Then the bed is subsequently desorbed by blowdown and purge steps in the cycle losing less light component. (Grande Carlos Adolfo, et al., 2008)The process can also have co-current or counter-current pressurization. (Cavenati S, 2005).

Later on, the equalization step, was introduced in 1966 by Berlin, (Chahbani, 2010) allowing a higher recovery of the light species and energy savings. The equalization step can be managed using two adsorbent vessels and partially pressurizing a second vessel using the gas leaving form the vessel being depressurized.

The rinse step can be carried out at any pressure between the high pressure and the low pressure and mostly uses a fraction of the low-pressure product after compression. It consists in circulating, in concurrent mode, a gas enriched with the most absorbable constituents with the objective of driving off the least adsorbable compounds from the adsorbent and the dead volumes (Renou, et al., 2015).

The purge step is very important for efficient operation because it allows the major portion of the bed to be free of adsorbate and ready for the whole cycle to be repeated. A purified component flows in the reverse direction causing the strongly adsorbed components to be flushed back towards the bed entrance.

In multicolumn processes, pressure equalization steps are frequently used to reduce the energy consumption of the system. The application of pressure equalization steps is more common for processes including more than 4 beds. The combination of different adsorbents and pre-purification steps has also been introduced in some processes.

PSA cycle is a sequence of elementary steps such as pressurization, feed, blowdown, purge, rinse and pressure equalization. During the feed step of PSA, the first column is fed at a higher pressure. When the adsorbent can no longer adsorb the heavy compounds, the feed is directed to other column or ceased. The blowdown occurs when the first column is vented to the atmosphere (losing the adsorbed compounds) opening the bottom valve. Although this is not enough to regenerate the adsorbent and a purging step takes place when a recycled part of the enriched air from the other column enters the column following the pressure differential between both columns. A

pressurization (using the feed stream) step restores the overall pressure of the system in the column with the regenerated adsorbent. Despite the discontinuous exit, the feed stream is being employed continuously (but only the synchronization of the operation in both columns can assure the availability of purge gas to the other column). However a tank is required to be coupled for a continuous discharge.

2.5.3 PSA – applications

PSA technology has been used in various applications: hydrogen purification, air separation, OBOGS (on-board gas generation system), CO_2 removal, noble gases (He, Xe, Ar) purification, CH_4 upgrading (Rao, 2010)], n-iso paraffin separation, and so forth.

Hydrogen Purification is one of the more used application is for instance which can be performed with a PSA unit. So, the increase in the demand for this gas resulted in a strong motivation for the development of new PSA processes. The need for these to be economical processes promoted the development of multi-bed processes (Batta, 1971) (Yamaguchi, 1993)

2.6 Mathematical Solving Approach for Adsorption beds

Process Analysis usually requires a deep knowledge of the process, a mathematical model together with the selection of the thermodynamic method to estimate physical and other properties.

Considering PSA, temperature, pressure, velocity and composition profiles are considered in the models as a function of time and location within the bed. Two phases should be taken into account, the gas phase and the solid phase so that the convective-diffusive phenomenon of transfer between them can be modelled. The mass transfer between the two phases occurs due to diffusion, convection, and adsorption/desorption mechanisms. These mechanisms are intimately related to the equilibrium conditions. Normally the applications of Adsorption deal with non-isothermal systems and non-spatially constant velocity flows during the adsorption and purge operations, which may lead to numerically intensive solutions. (Oliver J. Smith, 1992). Partial differential equations, boundary and initial conditions are crucial to solve systems with adsorption beds, since these are normally a sequence of elementary steps which require appropriate boundary conditions. (A parametric study of layered bed PSA for hydrogen purification, 2008)

The development of specialised algorithms for solving bed models is required for simulation. These models are described by partial differential-algebraic equations (PDAEs) in space and time, derived from heat, mass and momentum balances plus transport and equilibrium equations.

Numerical techniques applied for PDAES have pros and cons, being based on different methods such as on the method of lines and difference schemes, and others use functional approximation method (collocation method).

Analysing and predicting the time transient behaviour of chemical and biochemical processes often focuses on dynamic simulation.

Most existing dynamic modelling and simulation tools are primarily suited to lumped parameter systems. The majority of processes in Chemical Engineering are intrinsically distributed by nature, meaning that their variables exhibit both temporal and spatial variations. Therefore, PDAEs arise in a large quantity of modelling applications and failure tests has to be honestly done. Numerical methods that are suitable to solve a given partial differential equaions PDEs system are not necessarily good to solve another accurately. These equations can lead to a completely different behaviour for each change in either one parameter or boundary condition. Solving PDAEs is inextricable difficult, and there is a lack of universally applicable solution methods in spite of all the effort applied in this subject. Among the numerical methods devoted to solve the PDE and PDEAs systems there are different types of methods.

Numerical methods may be based on: (Le Lann, et al., 1998)

- Method of lines (MOL) (An Introduction to the numerical method of lines Integration of Partial Differential Equations, 1977) (Schiesser, 1991)
- Finite difference Methods
- Weighted Residuals Methods
- Finite Element Methods
- Finite Volume Methods
- Adaptive Grid Methods
- Moving grid Methods

All of these methods show pros and cons in a numerical point of view and need to be applied correctly according to the problem in study:

The method of lines (MOL) can be used to solve the PDEs. This method allows solving partial differential equations (PDEs) in which all but one dimension is discretized. The equations are first discretized in space, leading to a system of ordinary differential equations (ODEs) or differential algebraic equations (DAEs), to which a numerical method for initial value ordinary equations can be applied. These are then integrated over time by standard routines. Being able to decouple space and time, high order accuracy can be obtained in each dimension. MOL involving space discretization, converts PDEs into sets of ODEs with respect to time with the main advantage of resulting in a sophisticated framework based on well-established methods.

General DOE or DAE solvers such as DASSL, DASOLV, RESEDA may be used for large set of DOEs or DAEs. However, especially when coarse spatial grids are required, these show main difficulties to control and estimate the impact of the space discretization error on the general numerical scheme.

Finite volume formulation's most interesting feature is that the resulting solution ensures a conservation of involved quantities such as mass, momentum and energy exactly satisfied over the whole computation domain,

and not only over any group of control volumes. However, this is not the reality when dealing with finite difference methods.

Finite element approach may handle problems with steep gradients and may deal with irregular geometric configurations since it takes advantage of the ability to divide domain of interest in elementary subdomains, namely elements.

Adaptive and moving grid methods consists in the use of a numerical method in which nodes are automatically positioned and so following or anticipating deep fronts. This seems to be the most promising method and it may be accomplished requiring only two basic strategies, the spatial redistribution of a fixed number of points and the local grid refinement. The original grid structure is preserved, but at the detriment of computer time and storage increase, which is the principal advantage of this method. In fact, as mentioned in Brenan et al. (1989), coding of such techniques can throw complex problems as discontinuities, frequent restarting of the numerical methods, being relatively difficult.

In fact, as referred by Oh (1995) in its attempts to propose a general framework under gPROMS®® (Barton, et al., 1994), dealing with modelling and simulation of combined lumped and distributed processes, the goal of constructing reliable software able to solve a wide spectrum of PDE/PDAE/IPDAE problems has not been fulfilled yet, looking in fact unreachable in the nearest feature.

Some of the reasons are that they have a considerable freedom of formulation, some specific features of PDE problems require very specific treatment and there is none truly universal numerical method. A lot of effort has been put in two main directions, despite this pessimistic view. There are packages with specific domain designed to deal with a particular physical problem domain such as CFD (Computational Fluid Dynamics) and General-purpose packages such as PDECOL. (Le Lann, et al., 1998)

In previous Model simulation carried out in the gPROMS®-modelling tool, the method of lines (Schiesser, 1991) adopted consists of two steps:

1. The discretization of the continuous spatial domains into finite grid of points, resulting in a set of differential algebraic equations (DAEs), and

2. Integration of the DAEs over time with DASOLV integrator based on backward differentiation formulae (BDF).

This method minimizes an integral error efficiently by this DAE integration technique, that being its main advantage. Cantered finite difference method (CFDM) in the context of the MOL is used for the discretization. (Le Lann, et al., 1998)

2.7 Mathematical Solving Approach for Periodic Adsorption Processes

To be able to understand the dynamics of the systems, mathematical models are required to help predicting and explaining the separation results. However, these processes are quite particular since they do not operate in steady state conditions, but in cyclic steady state conditions (CSS) caused by periodic transient conditions with each bed repeatedly undergoing a sequence of steps. In each cycle concentration profiles change dramatically. The technical and economic performance of the operation can only be fully determined after this state is guaranteed.

This type of cyclic separation process, PSA is more advantageous over other separation options for middle scale processes. However, the development of automated tools for the design of PSA processes is a difficult task due to the complexity of the simulation of PSA cycles and the computational effort required to detect the performance at cyclic steady state. Due to the inherent complexity, variety, and computational effort to simulate these processes, the design and optimisation is still an experimental effort, despite the rapid growth in practical applications of PSA and the growing accuracy of bed models (Sircar, 2006)

The long-time behaviour of these processes needs to be known, and the convergence to the CSS especially with large capacity terms and slow kinetic terms, may take too much time. Most PSA processes are very complex and expensive and time-consuming when solved in an accurate way that agrees with the reliability needed for industrial design. In order to achieve the CSS, series of complete cycles are simulated until the characteristics of the system repeat in a periodic way. To do this, successive substitution is required in such a way that it reduces the number of simulations before converging to the CSS. Being a very complex process, same cases are difficult to by applied for different PSA systems

In order to optimize these type of processes, CSS needs to be simulated. In fact, the dynamic simulation may require more than thousands of cycles, enforcing the use of over-simplified models. Often, the mass transfer model is based on the linear driving force (LDF) approximation, which eliminates the need to describe concentration profiles within the solid. Indeed, for parabolic concentration profile within the particle, the full diffusion model and the LDF approximation are equivalent, however this condition holds for slow cycles. Unfortunately, the LDF approximation fails to describe the dynamics of fast cycle operations, required to decrease capital and operating costs. When the diffusional time constant R2/D, where R is the radius of the particle and D the diffusivity, is large with respect to the cycle time, a PSA cycle is fast. This type of cycles lead to a flat concentration profile at the core of the particle but at the same time result in quick variations in the outer region of the adsorbing particle and so a detailed diffusion model which requires the solution of coupled nonlinear partial differential and algebraic equations in time and space is required.

The models need to be approached in a different mathematical way that simplifies the problem and makes it easier to handle, and faster to converge. However they still have to be realistic, meaning they may be used in various configurations such as abrupt changes, start-up and shutdown configurations. Furthermore, these should be used for simulation, including parameter estimation, even in the near future for fast real time computing. Which may involve the requirement of state events detection, treatment of internal discontinuities (change in model mode:

change in the number phases in the systems) and external discontinuities due to deep change on operation parameter or even on feed.

Few of the studies have considered fast cycles and multiple-bed operations, which are necessary features to reduce the costs of the operation and increase efficiency. Advances both in simulation and optimization were done (Joankin, et al., 2012). In previous works, Broyden's method was used to accelerate the convergence (Smith and Westerberg, 1992) and Newton's method was used by (Croft and Levan, 1994). A simultaneous discretization of both spatial and temporal domain was used to calculate a CSS of two different PSA systems (Nilchan and Pantelides, 1998). However, many studies have addressed the design of PSA cycles via optimisation (Nilchan and Panthelides 1998, Biegler et al. 2004, Cruz et al. 2005). Moreover, the multi-objective optimisation of PSA operations has been addressed by few studies. This type of optimization would help identify the trade-offs between the different aspects of the performance. However, taking into account the large number of degrees of freedom, it is desirable to allow the enhancement of the performance of PSA cycles, and subsequently to expand the application of the process, which is possible with a mathematical programming approach to the optimisation of PSA processes. (Nilchan and Panthelides 1998, Biegler et al.2004)

Performance at CSS can be detected using different methods. A very computationally demanding method is Successive Substitution. It is a method where starting from a given initial state and by repeated dynamic simulations CSS is reached. Alternatively, the equations describing the system can be simultaneously discretised in the space and time while the periodicity condition imposed as a constraint ((Ko, et al., 2002), Nilchan and Panthelides 1998). This approach can suffer, however, from convergence issues (Nilchan and Panthelides 1998). In place of this method, detecting CSS can be treated as an optimisation problem itself ((Ding, et al., 2002), Jiang et al. 2003).

In conclusion, some of the solution strategies may include PDE discretization, CSS convergence acceleration, sensitivity evaluation and optimization.

2.8 Mathematical formulation of Periodical Adsorption Processes and CSS

Some of the approaches referenced in the previous chapter are now explained. To solve these problems, the models are often discretized spatially and even temporally, so that a differential algebraic equations system, f, resulting from a spatial discretization of the bed model describing a periodic adsorption process can be considered. A state variable is one of the set of variables that are used to describe the mathematical "state" of a dynamical system. The state of a system describes the system at a certain point, and so, it is enough to determine its future behaviour in the absence of any external forces affecting the system. In this system x are the state variables, y are the algebraic variables, and u are the input variables:

$$f(\dot{x}, x, y, u, t) = 0 \quad \textbf{(1)}$$

where f: $\mathbb{R}^{n_x+n_y+n_u} \rightarrow \mathbb{R}^{n_x+n_y}$.

The sum of the time of all the steps that consist in one cycle, a forced periodic operation can be taken into account over a cycle time T_c. This leads to a forced periodic system, in which the inputs (u) cycle continuously over a period T_c. In fact, this is what happens when valves are opened or closed during a cycle of a periodic process, inputs to the system cycle vary over a period.

$$u(t) = u(t + T) \quad \forall t \quad \textbf{(2)}$$

In this type of processes, the final state of one cycle is a function of the initial state of the cycle.

$$x(T) = \phi(x(0)) \quad \textbf{(3)}$$

Therefore, function ϕ is a mapping from the initial state to the final state, with ϕ: $\mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_x}$.

At **cyclic steady state,** the final state of a system after a cycle is identical to that at start of the cycle.

$$x(T) = x(0) \quad \textbf{(4)}$$

According to the former, reaching CSS is done finding values x^* such that:

$$x^* = \phi(x^*) \quad \textbf{(5)}$$

Or even, with the nonlinear system R: $\mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_x}$:

$$R(x) = \phi(x) - x \quad \textbf{(6)}$$

In CSS x^* is the solution of $R(x^*) = 0$, meaning that the difference between the system state at the beginning and end of one cycle, or residuals must all be very close to zero (with very small δ).

$$\|R(x^k)\|_\infty < \delta \quad \textbf{(7)}$$

2.9. Successive substitution

The most used method is still the direct method so called successive substitution, which consists in simulating cycles, using the state at the end of one cycle as the beginning of the next one until the state variables at the start of a cycle $k + 1$ are equal to the ones at the end of cycle k, meaning:

$$x^{(1)} = \phi\left(x^{(0)}\right) \text{ and } x^{(k+1)} = \phi\left(x^{(k)}\right) \quad \textbf{(8)}$$

x^k are the state values at the start of cycle k.

In this way, the residual can be defined as before. Despite being the most used method, it displays very slow convergence for some examples.

2.10 Acceleration

Convergence can be enhanced by applying Wegstein method only every so many iterations instead of each iteration. For instance, applying this method after some Successive Substitutions, doing new Successive substitutions and then applying it again. This iteration method is the default method in *Aspen Plus.* However, methods such as the direct method, Broyden's method and Newton's method can be used.

The main difference is that Wegstein method works each variable at a time, while the Broyden's method works on all the variables at once (resorting to a matrix that is adjusted iteration by iteration). (Finlayson, 2014)

2.10.1 Wegstein method

The Wegstein or secant convergence method is used for convergence and is very easy to implement in a computer program. It requires the objective function in the form $x = g(x)$ and two initial estimates of the solution $x^{(1)}$ and $x^{(2)}$ and the function is evaluated at these two points g_1 and g_2. The third point is calculated where the secant intersects the line at $x = g(x)$ and if g_3 is sufficiently close to $x^{(3)}$, $|\, g_3 - x^{(3)}|$ is very close to zero, the solution has been found.

According to literature, this method does not consider interactions between variable, so it does not work well when variables are strongly dependent. (Aspen Technology, 1998)Wegstein method can only be used for tear streams and it is usually the quickest and most reliable method for tear stream convergence. A tear stream is any stream that "opens" a loop. It is a stream that can be updated until two consecutive iterations stay within the

specified tolerance. In fact, this method is commonly used in the commercial flowsheet simulators to converge material and energy balances for flowsheets involving material recycle.

It extrapolates the direct substitution iteration. It is the default method for Aspen Plus tear stream convergence. It can be applied to any number of streams simultaneously. This method is explained in more detail in chapter 3.1.

2.10.2 Newton method

Newton method can also be applied to this type of problems. In fact, with $\mathrm{R}(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x}) - x,$ Newton method can be used to update the solution x^{k} by a Newton correction $\Delta\mathrm{x}^{\mathrm{k}}$. However a Jacobian needs to be calculated, or the derivative of R with respect to the state:

$$\mathrm{J}^{\mathrm{k}}\Delta\mathrm{x}^{\mathrm{k}} = -\mathrm{R}\left(\mathrm{x}^{\mathrm{k}}\right) \quad \textbf{(9)}$$

$$\mathrm{x}^{\mathrm{k+1}} = \mathrm{x}^{\mathrm{k}} + \Delta\mathrm{x}^{\mathrm{k}} \quad \textbf{(10)}$$

ϕ is the result of the DAE integration over one cycle. However considering the following:

$$\mathrm{J}^{\mathrm{k}} = \frac{\partial \mathrm{R}}{\partial \mathrm{x}}\Big|_{\mathrm{x}^{\mathrm{k}}} = \frac{\partial \phi}{\partial \mathrm{x}}\Big|_{\mathrm{x}^{\mathrm{k}}} - \mathrm{I} = \mathrm{X}^{\mathrm{k}} - \mathrm{I} \quad \textbf{(11)}$$

$\frac{\partial \phi}{\partial \mathrm{x}}\Big|_{\mathrm{x}^{\mathrm{k}}}$ is the partial derivative of the states (in time throughout the cycle) to the initial conditions, meaning that $\mathbf{X}^{\mathbf{k}}$ is the sensitivity of the states to the initial values: $\mathbf{X} \equiv \frac{\partial \mathrm{x}}{\partial \mathrm{x}_0}$. To solve for $\mathbf{X}^{\mathbf{k}}$,a matrix of size N_x by N_x, the sensitivity equations (given by the following equations) must be integrated.

$$\mathrm{f}_{\dot{\mathrm{x}}}\dot{\mathrm{X}}^{\mathrm{k}} + \mathrm{f}_{\mathrm{x}}\mathrm{X}^{\mathrm{k}} + \mathrm{f}_{\mathrm{y}}\mathrm{Y}^{\mathrm{k}} = \mathbf{0} \quad \textbf{(12)}$$

$$\mathrm{X}^{\mathrm{k}}(0) = \mathrm{I} \quad \textbf{(13)}$$

This method shows a quadratic convergence rate and the residual infinity norm has converged to very high accuracy for an RPSA example with 364 state variables (about 10^{-8}) after just 4 iterations (what should be compared to nearly 6000 successive substitution iterations) (Pattison, et al., 2014). A reasonable initial guess should, however be found, to assure that it will reliably converge to the solution. In order to do this, some successive substitutions are needed (for instance 10 iterations before implementing the Newton method). However, sensitivity equations have a lot of state equations (N_x^2) to be integrated, and in this type of processes,

after spatial discretization, hundreds of thousands of states result in more than million sensitivity equations. As expected, this can be very time consuming, in fact, Jiang Fox and Biegler reported that 90% of the computational time was spent integrating the sensitivity equations, meaning, 2 hours to calculate the Jacobian matrix for relatively small example with just 226 states. (Jiang, et al., 2004)

2.10.3 Newton Method variants

Newton method can also have variants, for instance there is the Discrete Newton method which approximates the Jacobian by finite differences. This method shows quadratic convergence and converges after a few Newton iterations, but requires hundreds of cycle simulations every iteration (since the system can have hundreds of states after discretization).

$$J_{ij} \approx \frac{R_i\left(x^k + \epsilon e_j\right) - R_i\left(x^k\right)}{\epsilon} \quad \textbf{(14)}$$

A simplified Newton Method uses for each iteration the initial Jacobian, showing linear convergence, requires hundreds of simulation to calculate the initial Jacobian, however the subsequent iterations require only one simulation.

$$J^0 \Delta x^k = -R\left(x^k\right) \quad \textbf{(15)}$$

2.10.4 State Profile Parameterization

Computing the set of initial conditions that result in the CSS can be done parameterizing the initial profile along the axial domain of the bed, z.

This method reduces the problem taking into account that the problem, its state variables, $x_{0,i}(z)$, can be described by independent variables, and that spatial variations of the initial states have parameterized profiles of the following form:

$$x_{0,i}(z) = k_{a_i} + \frac{k_{b_i}}{1 + \exp\left(-\frac{z - k_{c_i}}{k_{d_i}}\right)} \quad \textbf{(16)}$$

Therefore, parameters: k_a, k_b, k_c, k_d need to be solved to determine CSS. However, this method is quite specific problem dependent and not generalizable, because it only approximates accurately the profiles of the problem to which it was applied. (Ko, et al., 2003) (Ko, et al., 2005)

2.10.5 Complete Discretization Approach

Discretization of spatial domain of the periodic process, $z \in [0, L]$ and temporal domain $t \in [0, Tc]$, leading to a set of algebraic equations including the periodicity conditions $x(z, T) = x(z, 0)$ or the CSS constraint that can solved using Newton type method. All Jacobian elements available analytically using this method, leading, however to a very large system. Potentially significant error in temporal discretization. (Nilchan, et al., 1998)

2.10.6 Jacobian-Free Newton-Krylov Method

Jacobian-free Newton Krylov (JFNK) method can be used for computing the CSS condition. It uses an iterative linear solver for the Newton steps, the Generalized minimal residual method (GMRES). This method does not require the Jacobian matrix since it only requires the product of the Jacobian with various vectors, being Jacobian-free. (Knoll, et al., 2004)

This method solves a linear system of type $Au = b$ and considers that the linear residual at iteration j is:

$$r_j = Au_j - b \quad \textbf{(17)}$$

Then, approximates the solution u_j with linear combinations of matrix-vector products, for instance $u_3 \approx \alpha_1 r_0 + \alpha_2 A\, r_0 + \alpha_3 A^2 r_0$. At each GMRES iteration coefficients, α are chosen to minimize the 2-norm of the residual:

$$\min_{\alpha} \left\| Au_j - b \right\|_2 \quad \textbf{(18)}$$

In order to compute CSS of a periodic adsorption process, the same nonlinear system is required:

$$R(x) = \phi(x) - x = 0 \quad \textbf{(19)}$$

In this system, the Newton correction is a linear system of equations, A is the Jacobian, u is the Newton update, and b is minus the residual, meaning:

$$J^k \Delta x^k = -R\left(x^k\right) \quad \textbf{(20)}$$

In order to solve this linear system, the evaluation of $\mathbf{J^k v}$ Jacobian-vector products is required, where **v** are various vectors. However, the product of the Jacobian with an arbitrary vector **v** can be obtained simply doing a single finite perturbation, and simulating one perturbed cycle to obtain the approximate Jacobian vector product, without ever computing the actual Jacobian.

$$J^k v \approx \frac{R(x^k + \epsilon v) - R(x^k)}{\epsilon} \quad \textbf{(21)}$$

Since GMRES solves a linear system $Au = b$ via an iterative method, the speed of convergence a major consideration that needs to be improved. In fact, the success of the JFNK method depends on how few iterations it takes to converge or on how rapidly the linear system converges. Each GMRES iteration requires the simulation of a perturbed cycle, which is the limiting step of the method. So, to fasten the convergence, the matrix A must suffer some changes. In order to help the convergence, the eigenvalues of A must be clustered away from the origin. The more clustered the eigenvalues, the more rapidly the linear residual will converge. This operation is called "Preconditionnig". It transforms matrix A with the following equations.

$$(AP^{-1})w = b \quad \textbf{(22)}$$

$$u = P^{-1}w \quad \textbf{(23)}$$

A new linear system that consists in solving this linear system for w arises. The system matrix is now multiplied by the preconditioner. Multiplying the preconditioner by the solution w allows finding u.

The perfect preconditioner is matrix P such that $P = A \Rightarrow AP^{-1} = I$. What means putting into writing, that the perfect preconditioner matrix turns all eigenvalues equal to 1, so it would simply be the inverse of the system matrix. However, this matrix is unavailable.

If the eigenvalues are randomly distributed around the origin, the results show a very slow convergence. In fact, from previous work, (Pattison, et al., 2014) reported that for an RPSA example with on adsorption bed and 364 state variables, the residual does not decrease much after 100 GMRES iterations. However for eigenvalues clustered tightly around a real value of 2, the result is a very fast convergence rate of the linear residual as it converges to an acceptable accuracy in about 10 iterations. In conclusion, Preconditioning can influence the clustering of the eigenvalues, and consequently, the convergence rate of GMRES.

In order to find the Jacobian, there is the Preconditioned Newton correction:

$$(J^k P^{-1})w = -R(x^k) \quad \textbf{(24)}$$

$$\Delta x^k = P^{-1}w \quad \textbf{(25)}$$

Now, the considered linear system required for the JFNK method is the Jacobian times the preconditioner times a vector w equals the residual. Then, the Newton method update is calculated by multiplying the vector by the preconditioner, after the linear system is solved for w.

Preconditioned Jacobian-vector products can be obtained simply multiplying the arbitrary vectors v by the preconditioner and then simulating the perturbed cycle:

$$JP^{-1}v \approx \frac{\left[R\left(x^k + \epsilon P^{-1}v\right) - R\left(x^k\right)\right]}{\epsilon} \quad \textbf{(26)}$$

Then, P^{-1} is chosen to be such that $JP^{-1} \approx I$, however, the Jacobian still needs to be found and, in fact, given a sequence of points x_j, $j = 0, 1, \ldots n$ and the corresponding values of some functions $R_j \equiv R(x_j)$, Jacobian approximations can be constructed $A_j \approx J_j = \frac{\partial R}{\partial x}(x_j)$. This means that an approximation of the Jacobian by considering a sequence of states Δx_j and the corresponding values of the function evaluationsΔR_j. This is known ad Least Change Secant Updates and it considers how the residuals change as the points x change. Regarding that the quality of approximation improves gradually from one point to following, as the points change and the functions are evaluated, in the absence of any other information, starting with $A_0 = I$ is not a problem.

Therefore, defining $\Delta x_j = x_j - x_{j-1}$ and $\Delta R_j = R\left(x_j\right) - R\left(x_{j-1}\right)$, Jacobian approximation may be obtained via (Broyden, 1965):

$$P_j = P_{j-1} + \frac{\left(\Delta R_j - P_{j-1}\Delta x_j\right)}{\left\|\Delta x_j\right\|_2^2}\Delta x_j^T \quad \textbf{(27)}$$

In fact, P_j^{-1} can be obtained directly via Sherman-Morrison formula:

$$P_j^{-1} = P_{j-1}^{-1} + \frac{\left(\Delta x_j - P_{j-1}^{-1}\Delta R_j\right)}{\Delta x_j^T P_{j-1}^{-1}\Delta R_j}\left(\Delta x_j^T P_{j-1}^{-1}\right) \quad \textbf{(28)}$$

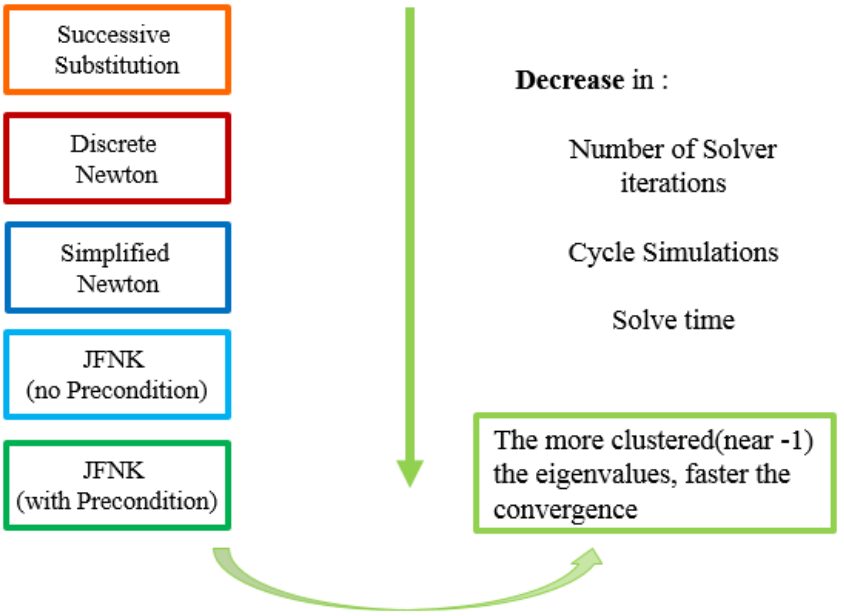

Figure 3 - Scheme with the various methodologies regarding different acceleration

Different methods were studied and the conclusion was that time should be invested in studying and investigating solvers with JFNK method (Pattison, et al., 2014)– which is, in fact one of the goals of this thesis.

2.11 Python language

Python is a programming language developed under an OSI-approved open source license, making it freely usable and distributable, even for commercial use. Python's license is administered by the Python Software Foundation. This makes Python a good software to be incorporated in other software packages, seen from the client perspective, because there is no special license needed to use it.

This language has a lot of applications and its syntax allows programmers to express concepts in fewer lines of code than would be possible in languages such as C++ or Java. It uses statements such as the ones described below:

- The *while* statement, which executes a block of code as long as its condition is true.
- The *def* statement, which defines a function or method.
- The *import* statement, which is used to import modules whose functions or variables can be used in the current program.
- The *print()* statement, which prints the variable inside the brackets. (Python, 2016)

This page was intentionally left blank.

3. CSS acceleration for simple gPROMS® models

A CSS acceleration was implemented for a gPROMS® model containing simple mathematical functions and for an adsorption bed gPROMS® model.

3.1 Wegstein method

Considering the objective function in the form $x = g(x)$ and two initial estimates of the solution $x^{(1)}$ and $x^{(2)}$, the function evaluated at these two points are g_1 and g_2. The third point is calculated where the secant intersects the line where $x = g(x)$. So, given $\mathrm{x}^{(\mathrm{k}+1)} = g\left(\mathrm{x}^{(\mathrm{k})}\right)$, the convergence rate depends on the gradient of $g(x)$. The gradient, s

$$\mathrm{s} = \frac{dg(x)}{dx} \approx \frac{g\left(\mathrm{x}^{(\mathrm{k})}\right) - g\left(\mathrm{x}^{(\mathrm{k}-1)}\right)}{\left(\mathrm{x}^{(\mathrm{k})}\right) - \left(\mathrm{x}^{(\mathrm{k}-1)}\right)} \quad \textbf{(29)}$$

So,

$$g\left(\mathrm{x}^{(2)}\right) = g\left(\mathrm{x}^{(1)}\right) + s\,(\mathrm{x}^{(2)} - \mathrm{x}^{(1)}) \quad \textbf{(30)}$$

Since, when convergence has occurred $\mathrm{x}^{(2)} = g\left(\mathrm{x}^{(2)}\right)$, then $g\left(\mathrm{x}^{(2)}\right)$, is substituted in equation (30) by $\mathrm{x}^{(2)}$:

$$\mathrm{x}^{(2)} = g\left(\mathrm{x}^{(1)}\right) + s\,(\mathrm{x}^{(2)} - \mathrm{x}^{(1)}) \quad \textbf{(31)}$$

With $q = \frac{s}{s-1}$, the Wegstein update is:

$$\mathrm{x}^{(2)} = g\left(\mathrm{x}^{(1)}\right)(1-q) + \mathrm{x}^{(1)}\mathrm{q} \quad \textbf{(32)}$$

Note: *q=0* in normal substitution. And that:

$$x_i^{(k+1)} = g_i\left(x_1^k, x_2^k, \dots, x_n^k\right),\ i = 1,2,\dots,n \quad \textbf{(33)}$$

With $\mathrm{x}^{(1)} = g\left(\mathrm{x}^{(0)}\right)$ and $\mathrm{x}^{(2)} = g\left(\mathrm{x}^{(1)}\right)$, from k=1, the gradients are computed:

$$s_i = \frac{g_i\left(x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)}\right) - g_i\left(x_1^{(k-1)}, x_2^{(k-1)}, \dots, x_n^{(k-1)}\right)}{x_i^{(k)} - x_i^{(k-1)}},\ i = 1,2,..,n \quad \textbf{(34)}$$

Update $x^{(2)}$ and subsequent estimates of solution:

$$x_i^{(k+1)} = g_i\left(x_1^{(k)}, x_2^{(k)}, \dots, x_n^k\right)(1 - q_i) + x_i^{(k)} q_i \quad \textbf{(35)}$$

Where

$$q_i = \frac{s_i}{s_i - 1}, \quad i = 1,2,\dots,n, \quad k = 2,3 \quad \textbf{(36)}$$

Table 1 - Convergence of Wegstein method accord to q.

Value of q_i	Expected convergence
$0<q_i<1$	Damped successive substitutions, Slow stable convergence
q_i=1	Regular successive substitutions
$q_i<0$	Accelerated successive substitutions Can speed convergence: may cause instabilities

Considering $x = g(x)$, a criteria for convergence of this method is that is that the $\left|\frac{\partial g(x)}{\partial x}\right| < 1$ for all x in the search interval. (Luyben, 1992) (Finlayson, 2014)

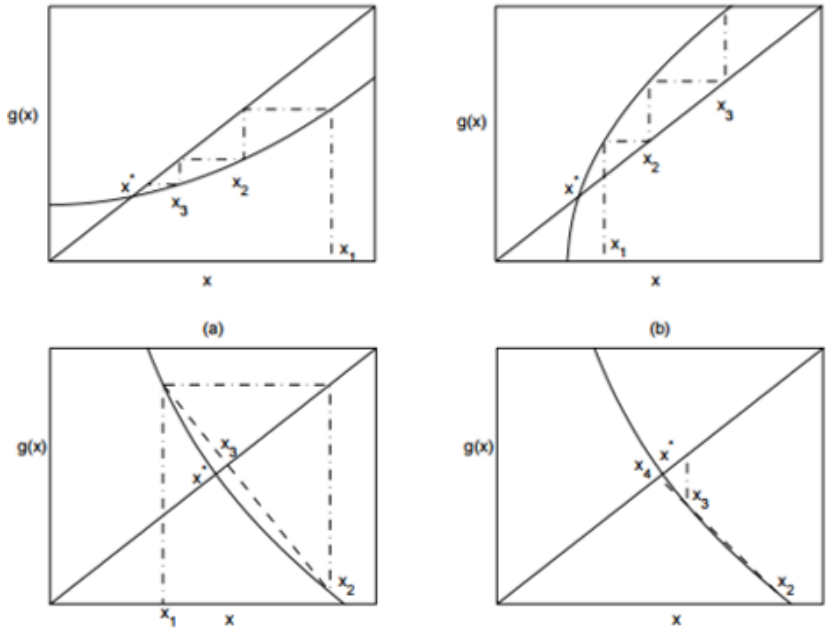

Figure 4 - Convergence and divergence to the solution in successive substitution (a) and (b) above. Wegstein method bellow (b), for convergence , compared with successive substitution (a).

Considering Figure 4, given a first point x_1, $x^*$is the solution of the equation. In the first case it converges, in the second one it diverges because $\left|\frac{\partial g(x)}{\partial x}\right| < 1$.

3.2 Architecture of the Python Program

A Python program was created in order to apply Wegstein acceleration to the convergence to the solution of simple functions in gPROMS® and for the convergence to the CSS of gPROMS® one bed RPSA gPROMS® ProcessBuilder example.

In order to do this, communication between Python and gPROMS® should be established. A Foreign Process Interface (FPI) available in gPROMS® allows the exchange data and other information between executing gPROMS® simulations and external software. This allows the interaction with an external software such as python. The interaction takes the form of a special set of elementary actions within the gPROMS® Task language.

Therefore, this communication happens at discrete time points throughout the simulation and the user can determine the frequency and content of the exchanges. The user can also determine how the information should be exchanged. In this case, it is done with text files.

3.3 Python program to communicate with gPROMS®

3.3.1 gPROMS® language

To assure communication between Python and gPROMS® using a Foreign Process Interface, the following tasks are important:

- **GET** tasks allow gPROMS® to 'get' the values of the variables presented in the task from a specific file/ place and to assign them to a specific variable in that point of the process when the task was executed.
- **SEND** tasks are able to 'send'/save/write the values (of the gPROMS® variables specified in the task) at the exact time the task is executed to a certain file/place.

In order to have the correct interaction between both programs, another task should call the above tasks in the correct point of the simulation. A first SEND is executed before the first cycle (including the initial state of the system). Then, a GET shall be executed before the cycle and a SEND after the cycle (with the final state of the system after that cycle).

In order to use the Text File FPI, the following line needs to be present in the SOLUTION PARAMETERS and in the SCHEDULE section of the PROCESS:

FPI : = "TextFileFPI::dummy –timeout #"

In the line FPI : = “TextFileFPI::dummy –timeout #” # can be any number. This line allows the usage of Text File FPI with the possibility of gPROMS® to abort the simulation after # seconds, if the necessary get text file is not available.

3.3.2 Python program architecture

The architecture of the program implemented in Python is for the acceleration algorithm. A more detailed description of the algorithm of the program shown in Figure 5.

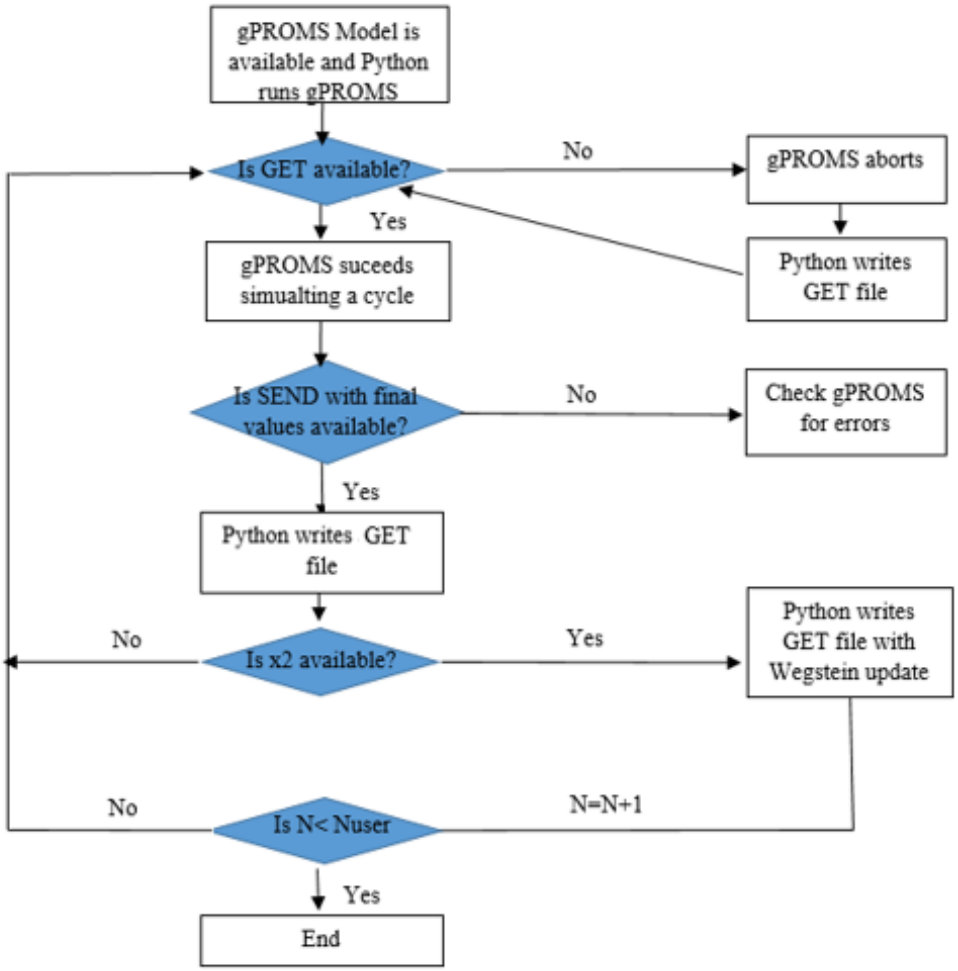

Figure 5 - Framework of Python and gPROMS® interaction to accelerate performance of gPROMS® models

Python uses the file exported from gPROMS® and after attempting to run gPROMS® the first time, it copies the send text file available containing the initial values. The next step reads the **SEND** text file and writes a **GET** text file equal to the first sent variables. After this step, the first **GET** text file is available and so gPROMS® is able

to simulate one whole cycle, writing a **SEND** text file containing the sent variables and their values and the end of the cycle. If the last step is done repeatedly, normal successive substitution is done.

The program is done in a way that the user only has to insert some parameters and the program applies the algorithm to all the values included in the **GET** and **SEND** tasks (described in section 3.3.1). This makes it possible to apply the algorithm for a simple function or for more than just one function at a time, like the state variables of an adsorption bed for instance.

The parameters to be inserted in Python interface are the following:

1. The number of Wegstein method iterations, **n**.

2. The location of the exported gPROMS® process.

3. The goRUN arguments.

The paths in the python program need to be set to the same folder where the gPROMS® process was exported to, only then goRUN can be executed correctly. .gORUN is a gPROMS application installed by default during the software installation that allows using a command line described in appendix 7.1, to run an exported gPROMS project.

To apply the Wegstein algorithm three first iterations are required in order to estimate the solution. So each time the algorithm is executed, the program executes gPROMS® process twice. The initial state is taken as first point and the final state of the first and second simulation are taken as second and third point respectively.

The state variables for the three points are saved in three different vectors, and the algorithm is implemented to those vectors resulting in a vector containing the solution. If the number of implementations specified by the user is higher than one, the previous solution is used as the first point for the new iteration. The program returns the state variables and the residual. A more detailed description of the algorithm of the program shown in is given in appendix 7.2.

3.4 Result with Wegstein acceleration

Wegstein method was applied to several functions and also to one bed RPSA gPROMS® ProcessBuilder example. In all cases, the value of q was determined by (36) and was never fixed or bounded. Some more details about this calculation procedure could be obtained at Appendix 7.4.

3.4.1 Simple mathematical Function

The first attempt to validate the Python program was applied to a simple function of the type $y = mx + b$. The convergence with this method was, in fact, accelerated. With $m = 0{,}9$ and $b = 1$, using as initial value $x(0) = 0$, the program converged to the solution with residual smaller to the order of 10^{-6} after 162 iterations, however with the Wegstein method it converged in only one iteration, which was a very promising result.

In order to study the convergence of this method, the program was tested for more equations of the type $y = mx + b$, with $b = 1$ and different slopes (m). The following graph represents the results.

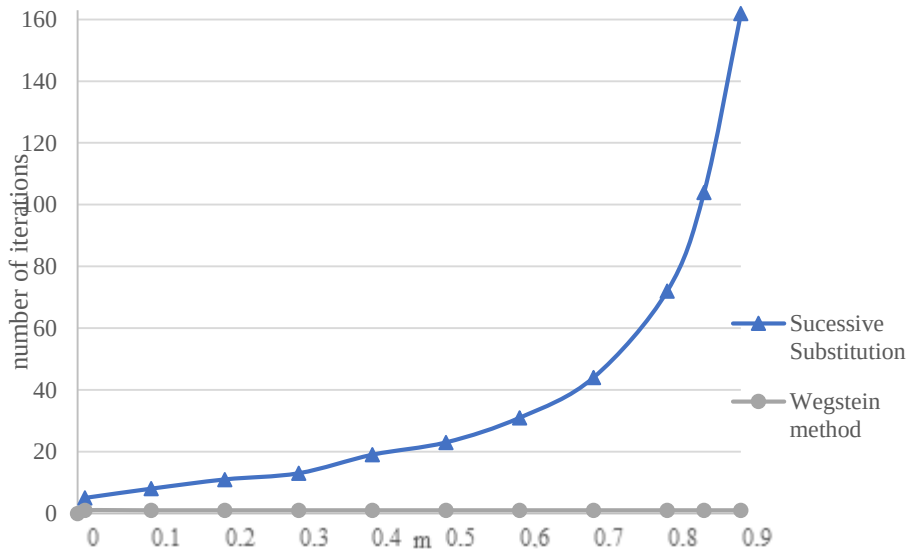

Figure 6 - Number of iterations that lead to convergence of linear equations regarding the function's slope

For $0 < m < 1$, an increase in m leads to a slower convergence of the succession $X(n+1) = m \times X(n) + b$, and the number of required iterations increases exponentially. However considering the Wegstein method, all these equations converge with just one iteration.

Another important result to consider is that for $m > 1$, the Wegstein method converges with one iteration while successive substitution diverges, meaning that the Wegstein method accelerates the convergence for all linear equations.

The following graph represents the results for an equation of type $y = mx^n + b$, with $m = 0.1$ and $b = 1$ according to the order of the equation (value of n) :

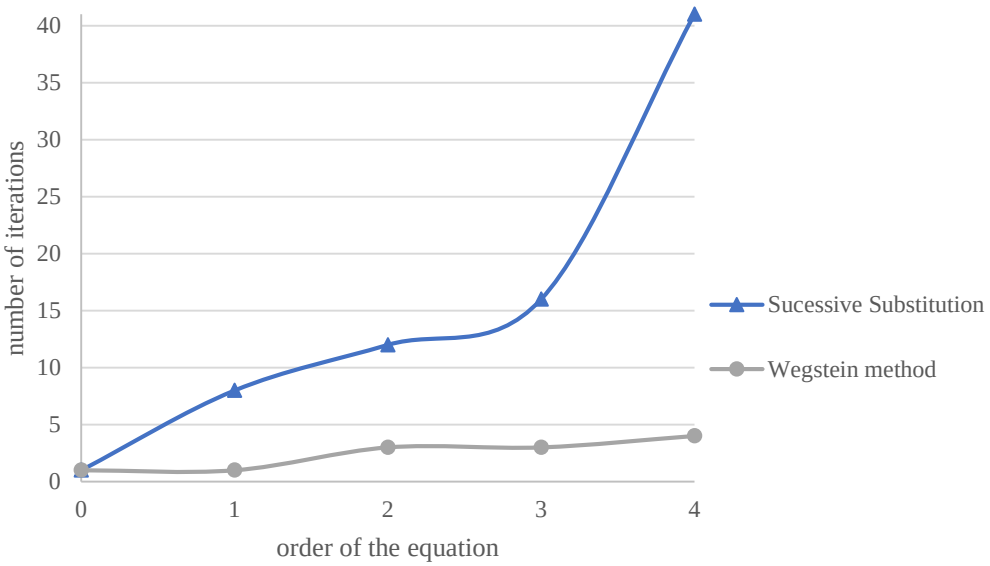

Figure 7 - Number of iterations that lead to convergence of non-linear equations regarding the function's order

All of this examples lead to accleration parameter q< 0, and so the Wegstein method should lead to accelerated successive substitutions. However it is possible that it causes instabilities.

For two studied equations of order 2 with $b = 1$, if m is -0.1, it takes only two Wegstein iteration. Moreover, a very similar equation, for $m = 0$, takes 2 iterations as well. For both cases the initial value was 1. In the case of $m = -0.1$, the algorithm leads to $0 < q < 1$, and with $m = 0.1$, q is negative, however, there is no difference in the number of iterations necessary to converge.

If the function of order 2 has m equal to 0.2, the acceleration parameter $q < 0$ for all iterations and so this method converges and the solution is found as follows, with $x(0) = 0$.

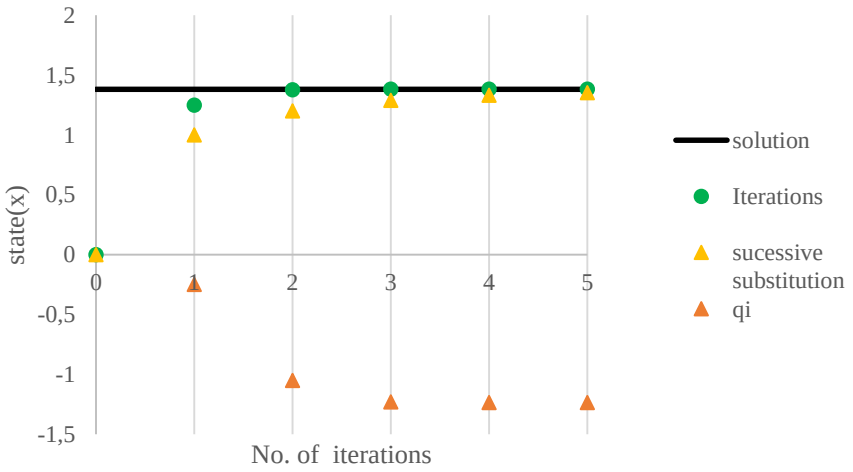

Figure 8 - Results for a non-linear second order equation that converges

In this case, $q < 0$ leads to accelerated convergence, however the results are not so different when compared to normal Successive substitution.

However, taking into account a cubic equation such as $y = x^3 - 6$, the solution diverges using successive substitution as can be observed in Figure 9:

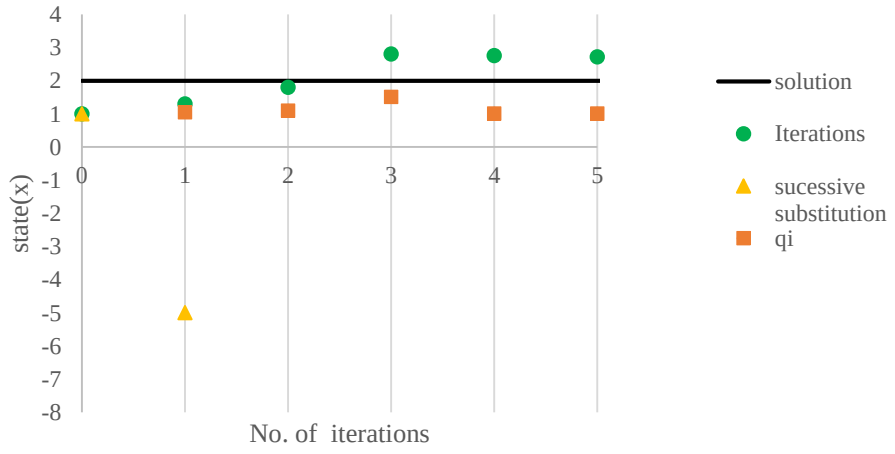

Figure 9 - Results for a non-linear second order equation that diverges

The values of the successive substitution are not in the graph because the values grow very fast as shown in the table 2. This happens because in this case the gradient of the function is bigger than 1, which leads to an acceleration parameter, $q > 1$, obtained for each iteration by (36) leading to the following situation:

Table 2 - Iteration values and parameters

No. of iteration	q_i for Wegstein method	State	
		Wegstein method	SS
1	1.05	1.3	-5
2	1.097954	1.799858	-131
3	1.509344	2.802879	-2248097
4	1.003243	2.760022	-1.1E+19
5	1.003652	2.715232	-1.5E+57
6	1.00415	2.668321	-3E+171

The Successive substitution method diverges almost instantaneously from the solution, while the Wegstein method oscillates 'near' the solution. This happens because the slope of the function in the searching interval is higher than 1.

3.4.2 Simplified Adsorption bed model in Process Builder

Then, it was applied to the one bed *RPSA* ProcessBuilder example, whose equations are described in section 4.2 (since these are the same as the ones in Model Builder example), and whose conversion into this platform is described in section 4.3.

This example was considered with five types of state variables, totalizing 85, showing a faster convergence, when the Wegstein method is applied to these variables. It is also necessary to take into account that one Wegstein iteration requires two successive substitutions, since it uses three points to estimate each iteration. However, even considering each of the necessary iterations to obtain enough points for each Wegstein acceleration, the convergence is, in fact accelerated. However, bounds of variables were not considered so this acceleration can lead to non-realistic results.

The following graph shows the evolution of the residual for the state variable with the method. This method shows linear convergence. In fact, only 4 Wegstein method iterations are enough to obtain the residual in the state variables considered of the order of 10^{-5}. This program took 41 seconds to display the result with four iterations.

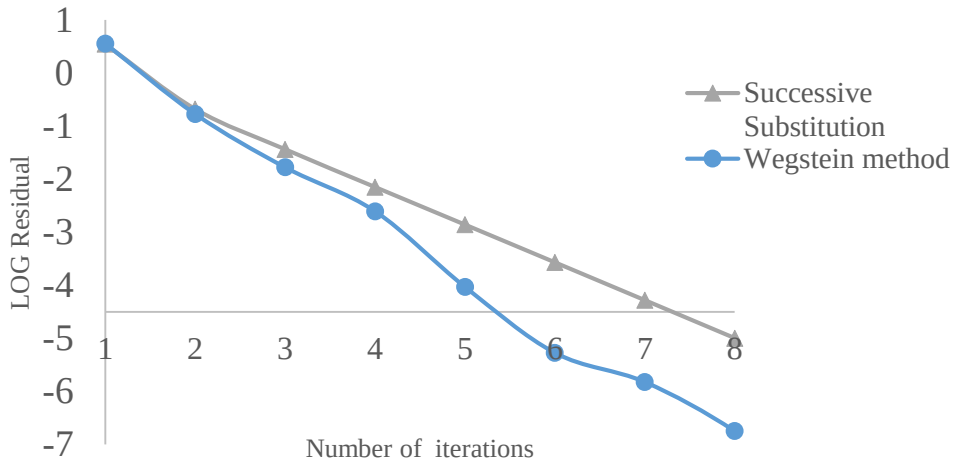

Figure 10 - Logarithm of the residual for each iteration using both Successive Substitution and Wegstein method

While performing the Wegstein acceleration, when the Residual reaches 10^{-6}, the Key Performance Indicators (KPI's) obtained after that last acceleration are very similar to the ones estimated with Sucessive Substitution. In conclusion, this method lead to accurate results, and is a good alternative to solve simple problems such as this one. In fact, this is was the only acceleration that was successfully applied to the one Bed RPSA ProcessBuilder example – the prototype cyclic solver described in section 4.4.1 was not yet robust enough to be used with this example.

4. CSS acceleration for a one bed RPSA model

This section includes the description of the one bed RPSA examples:

-The one developed in ModelBuilder, previously at PSE, is described in section 4.1 and 4.2, and accelerated with the prototype solver with JFNK method developed at PSE, in section 4.4.

- The one obtained after the conversion of the previous one into ProcessBuilder, presented in chapter 4.3. This example was created during the project of the present thesis is validated in section 4.3.2, and was not accelerated with the prototype solver developed at PSE, but with the Wegstein method implemented in Python described in Chapter 3.

4.1 Problem Description

The goal of this problem is to produce, from atmospheric air, oxygen-enriched air with a RPSA unit. The bed contains a single adsorbent, and the operation is done cyclically. Each cycle is composed by Pressurization and Depressurization, as described in Figure 10.

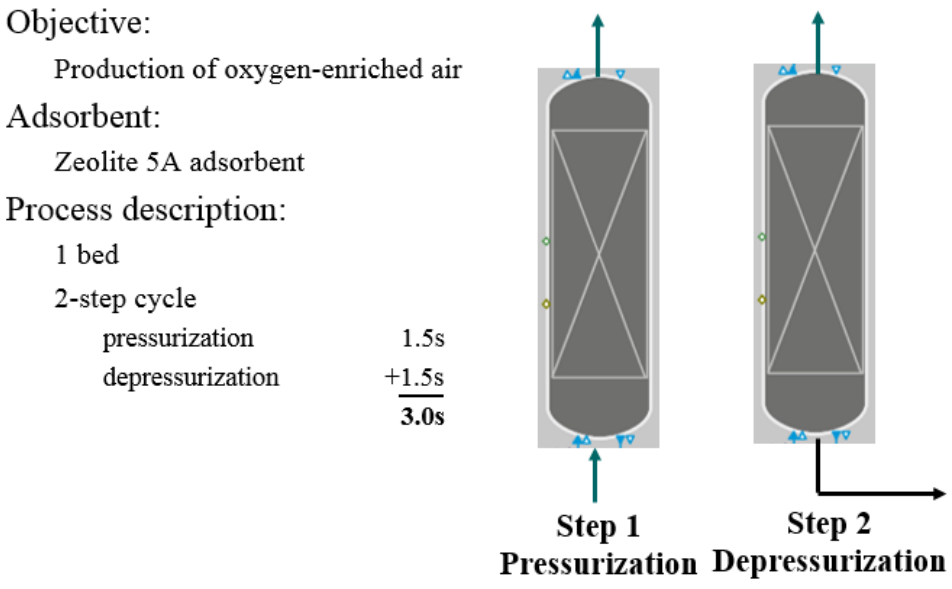

Figure 11 - Summarized description of the example

The example in question was developed at PSE, in gPROMS® ModelBuilder, and validated before being used in this thesis.

The existent model, developed in gPROMS® ModelBuider, considered an adsorption bed defined as a unit with some parameters. Amongst the parameters there were the number of components, bed length, density and cross-sectional area, bed and particle void fractions, Particle radius, diameter, pore diameter and tortuosity factor, gas viscosity, axial diffusivities for each component, adsorption isotherm gradients for each component, molecular weights for each component, ideal gas constant, feed conditions: feed pressure, temperature and composition, product conditions volumetric flow and pressure (atmospheric) as well as waste pressure. The considered distribution domain was considered as Axial. The table including the specifications is in Appendix 7.3.

In addition, several variables were used such as Gas phase concentrations, Solid phase concentrations, Equilibrium solid phase concentrations, Superficial gas velocity, Pressure, Temperature, Mass transfer coefficients, Amount of material fed to the column, Amount of material escaping in the product stream, Amount of material escaping in the waste stream, Product purity and Product recovery.

In this example, n, the number of components, equals 2, since there are only two components, oxygen and nitrogen.

4.2 Mathematical description of the bed model

In order to structure the model to simulate this process, the following model assumptions were considered:

1. The operation of the bed is isothermal.
2. There are no radial variations in the bed.
3. The fluid phase follows Ideal Gas Law.
4. The adsorption bed parameters (i. e. bed void fraction, bed bulk density, and particle size) are uniform and constant.
5. The flow in the bed in axially dispersed.
6. The axial pressure drop can be described in a satisfactory way by Darcy's law.
7. The mass transfer rate is described by a linear driving force model.

The model of adsorption bed for a mixture separation of n components model could be described by the following equations:

Considering $i = 1$ to the number of components (n), the component mass balance is done for each component inside de bed, not including the boundaries.

$$\varepsilon_{tot}\frac{\partial C_i(\boldsymbol{z})}{\partial t} = -\frac{\partial C_i(z)u}{\partial z}\frac{\partial}{\partial z}\left(D_i\frac{\partial C_{i(z)}}{\partial z}\right) - \rho_{bed}\frac{\partial q_i}{\partial t}, \quad \textbf{(37)}$$

With,

$$\varepsilon_{tot} = \varepsilon_{bed} + \varepsilon_p\,(1-\varepsilon_{bed}) \quad \textbf{(38)}$$

The ideal gas law includes the end bound, z=L:

$$\frac{P}{R\,T} = \sum_{i=1}^{n} C_{i,}(\boldsymbol{z}) \quad \textbf{(39)}$$

Meaning that filling the bed with atmospheric air equals:

$$P_{atm} \times Y_{feed} = \sum_{i=1}^{n} C_i\,(z)\ \times R \times T_{feed} \quad \textbf{(40)}$$

For the pressure drop, Darcy's pressure drop equation can be considering for the steady state momentum balance of gas flow at low velocity through a packed bed:

$$-\frac{\partial P}{\partial z} = 180\frac{\mu u(1-\varepsilon_{bed})^2}{{\varepsilon_{bed}}^3 {d_p}^2} \quad \textbf{(41)}$$

As the Equilibrium isotherm, an approximated Langmuir isotherm, a simple linear isotherm, is taken into account for each component and the whole bed:

$$q_{eq,i} = m_i\,P_i = m_i\,C_i(z)\,R\,T \quad \textbf{(42)}$$

The adsorption rate equation considered is linear for all bed and each component.

$$\frac{\partial q_i}{\partial t} = D_i\frac{\partial q_{i(r)}}{\partial r}\Big]_{z,r=rp} = k_i(q_{eq,i} - q_i(r_p)) \quad \textbf{(43)}$$

This equation means that the concentration of specie i in the bed is proportional to the difference between the adsorbed amount at equilibrium $q_{eq,i}$ in the macropores and the current value of the adsorbed phase. K_i or k_{LDF} is

the mass transfer coefficient and is a proportionality constant. Heat of adsorption is considered to be zero. Starting with the adsorbent at equilibrium with gas phase equals:

$$\frac{\partial q_i}{\partial t} = 0 \quad \textbf{(44)}$$

Where the mass transfer coefficient is given by the following equation:

$$k_i = \frac{15\ \varepsilon_p^2 (1-\varepsilon_b)}{r_p^2 \rho_{bed}\ m_i R\ T\ \tau} \quad \textbf{(45)}$$

The amount of material fed to the column, amount of material escaping in the product stream and of material escaping in the waste stream were integrated in order to calculate purities and recoveries.

Table 3 - Equations to obtain the amount of material fed and escaping from the adsorption bed

Pressurization	**Depressurization**
$\boldsymbol{M_{feed,i} = \int_t^{t+T_c} u(0)\ C_i(0)\ \ dt}$ (46)	$M_{feed,i} = 0$ (47)
$\boldsymbol{M_{waste,i} = 0}$ (48)	$M_{waste,i} = \int_t^{t+T_c} u(0)\ C_i(0)\ \ dt$ (49)
$\boldsymbol{M_{product,i} = \int_t^{t+T_c} u(Bedlength)\ C_i(Bedlength)\ \ dt}$ (50)	

In order to operate the column, the operation mode Pressurization and Depressurization were defined, and was set to be Pressurization as default.

The velocity is defined for the boundaries and the Boundary conditions were considered both for the feed and product end of the bed for Pressurization and Depressurization.

Table 4 - Boundary conditions for operation steps

Pressurization	Depressurization
$\boldsymbol{P_{feed \times Y_{feed}(i)} = C(i,0) \times R \times T_{feed}}$ (51)	$\frac{dC(i,0)}{dz} = 0$ (52)
$\boldsymbol{P_0 = P_{Feed}}$ (53)	$P_0 = P_{Waste}$ (54)

The variables Purity and Recovery were assign to zero at the beginning of the simulation, so that they can be calculated after each step.

The initial conditions were specified in the process: filling the column with air at atmospheric pressure, starting with adsorbent at equilibrium with the gas phase and counters of the feed, product and waste were set to zero (10^{-6}) as well. The time of a cycle was considered to be 3 seconds, with pressurization and depressurization steps having the same time length.

The Results of this model are shown in section 4.3.2., with the validation of the one bed RPSA ProcessBuilder example.

4.3 Conversion of RPSA to gML

This section includes the procedure done to convert the existent example from the gPROMS® ModelBuilder platform (whose description is in sections 4.1 and 4.2) into ProcessBuilder, as well as its validation with the former.

4.3.1 Procedure

The example in gPROMS® ModelBuilder to obtain oxygen from a stream of air, containing an adsorption bed with one adsorbent was considered and converted to a gML that could be included in the libraries of the recently launched in ProcessBuilder package.

The procedure is described in appendix 7.5. Several main units were considered in this example and converted (shown in table 5) to obtain the flowsheet presented in Figure 11.

Table 5 - Units and corresponding gML libraries

Units in ModelBuilder	gML used
Column	Adsorption_bed_gML
Feed	Source_material_reversible_gML
Product	Source_material_reversible_gML
Waste	Source_material_reversible_gML
Feed_Valve	Valve_reversible_gML
Product_Valve	Valve_reversible_gML
Waste_Valve	Valve_reversible_gML
Scheduling	Scheduling_gML

In this conversion action two more units were added in order to replicate the example, since the volumetric flow at the product end of the bed was set to be constant:

- Stream_analyzer_reversible_gML AS Stream_analyzer_reversible_gML - with the reporting (Overall) set to Volumetric flowrate.
- Adj_spec_gML AS Adj_spec_gML

The RSPA gML to run in ProcessBuilder environement has the column pressure drop specified by the Darcy's pressure drop correlation.

In the Adsorption bed topology tab, specifically in the Fluid (mass transfer) part, the Mass transfer from fluid to adsorbent(s) was set to be 'LDF (Linear Driving Force). The driving force was considered based on the Solid concentration, the dispersion coefficient was specified and Mass transfer coefficient correlation set to be Customized (only in this way the process will use the customized correlation defined as a model called mass_transfer_coefficient_adsorption_custom_gML).

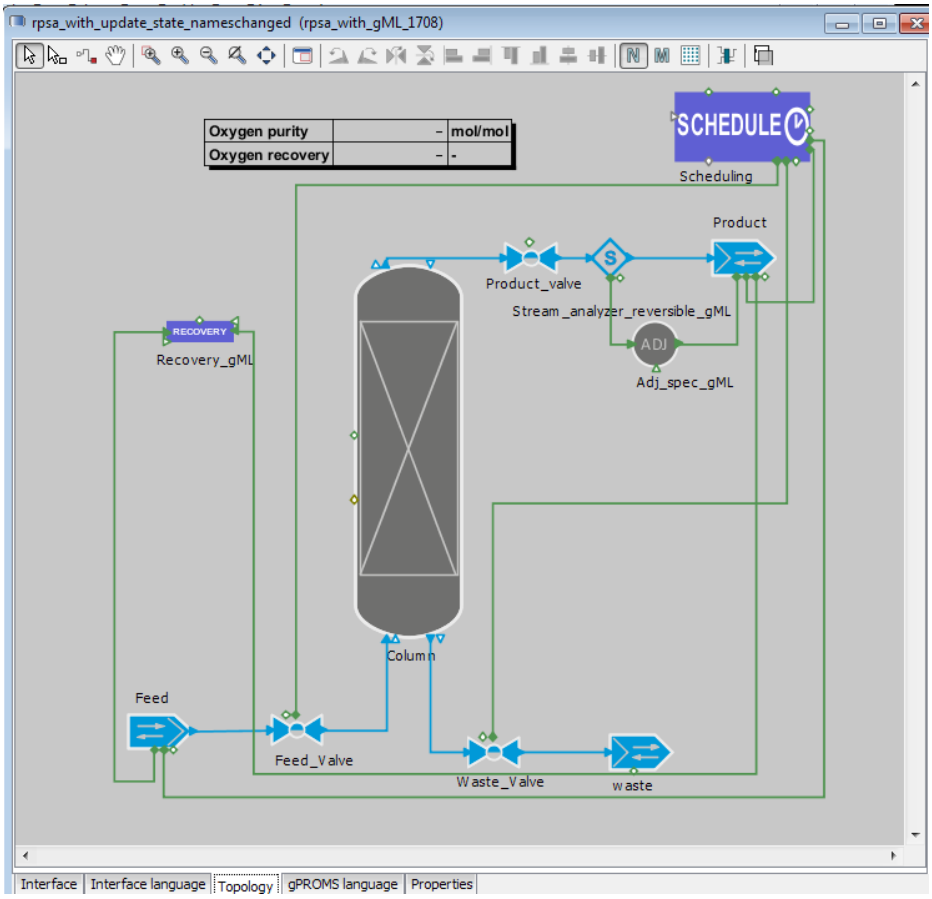

Figure 12 - Flowsheet of the RPSA example in Process Builder

For replicating the Langmuir isotherm equation in gPROMS® ProcessBuilder, it was inserted in a model called isotherm_section_custom_gML. This procedure in Process builder was developed in two steps:

1) In the topology tab and in the isotherm part, needs to be changed to Custom.

2) The mass transfer coefficient equation was inserted in a model called mass_transfer_coefficient_adsorption_custom_gML. Since the operation was considered to be isothermal in the model: $T(z) = T_{feed}$, the correspondent specification is done in the Fluid (heat transfer part) , setting Thermal operation mode as Isothermal.

The amount of material fed and escaping from the adsorption bed is already defined in the gML libraries as the holdup in molar fraction.

In order to assure the operation, the openings and closures of the valves are managed by a schedule model.

For the schedule, the specifications for the two operation steps (Pressurize and Depressurize), taking into account that 1 corresponds to a fully opened valve and 0 to the opposite, were done accordingly to the table in figure 13:

Units

Steps	Feed_Valve	Waste_Valve
Pressurize	1	0
Depressurize	0	1

Figure 13 - Valve positions specified in the Schedule

This is very important to be well assign for a PSA process. In fact, the results depend a lot on the valve behavior along time, meaning step durations and cycle sequence for instance.

All the valves were set in the Dynamics tab to be flow-driven. Both the feed and purging valves' operation is assigned by connection. This allows the schedule to set the stem position of the valve. Only the product valve, which is always opened has the stem position specified in the dialogue box. These valves also differ from the other valve in other aspects. In the last valve, the Main Outlet Specification is the Pressure drop, while in the others it is the flow coefficient.

This simulation relies on a standard solver for differential-algebraic equations, DASOLV, which based on variable time step/variable order Backward Differentiation Formulae (BDF) and is efficient for a wide range of problems.

The solution parameters are shown in figure 14, including the necessary line for the FPI usage previously explained in chapter 3.

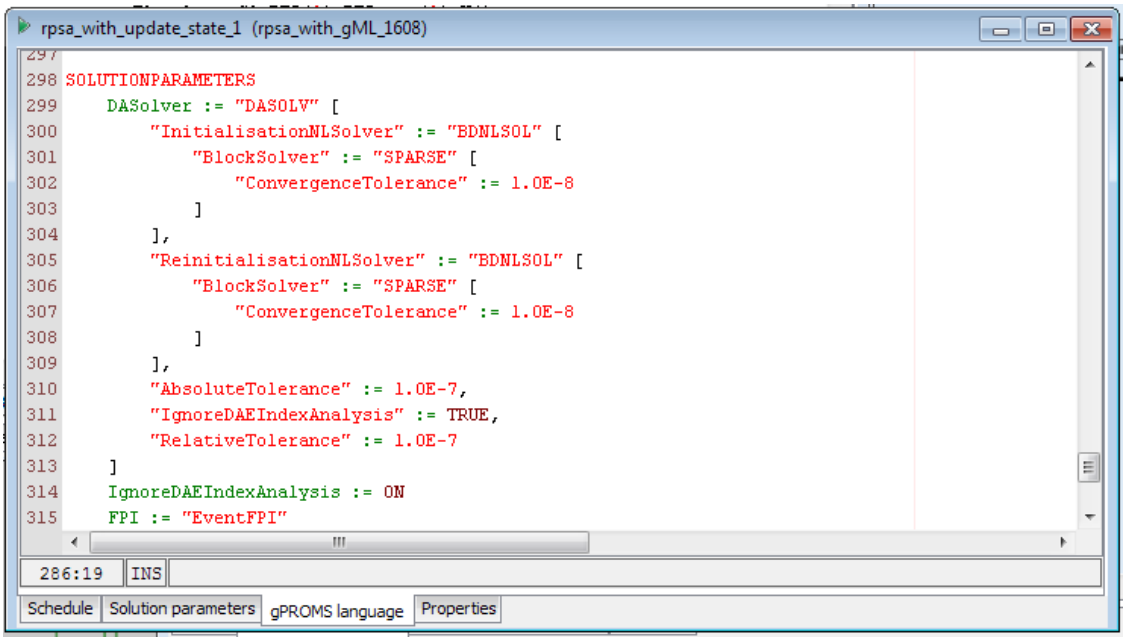

Figure 14 -Solution Parameters of the one bed RPSA gPROMS® ProcessBuilder

4.3.2 Validation

The design and optimization is meaningful in CSS and some specific features of PDE problems require very specific treatment leading to different results depending on the solving method.

The model implemented in gPROMS® ProcessBuilder was compared with the on previously created in gPROMS® Modelbuilder. In order to compare the results of each model two Key Performance Indicators (KPI's), purity and recovery were defined. On the cyclic steady state some differences are still found in both KPIs, however the results are very similar.

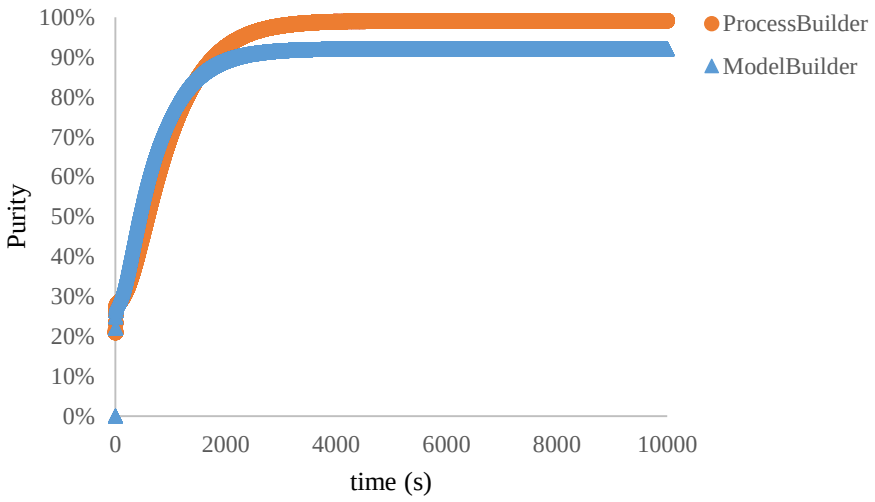

Figure 15 - Purity of the obtained Product with Process Builder and Model Builder

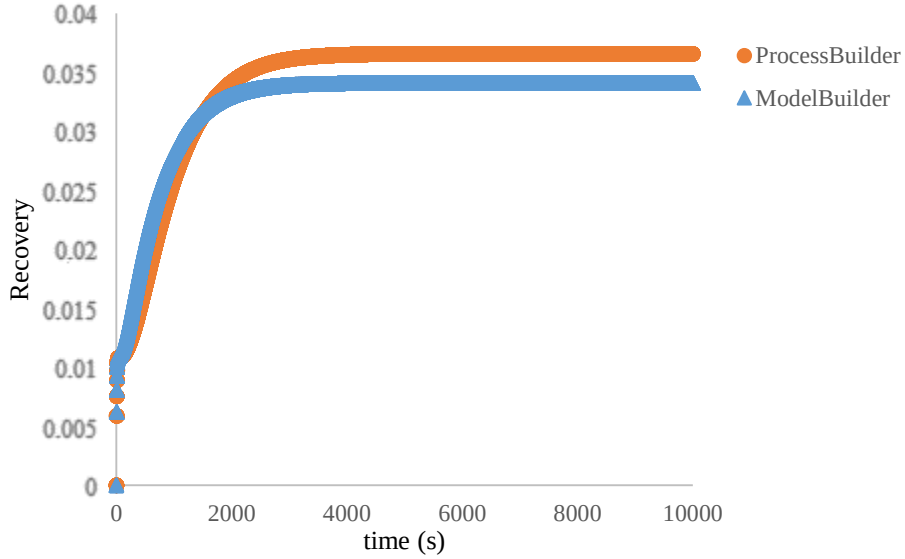

Figure 16 - Recovery of Oxygen with gPROMS® ProcessBuilder and ModelBuilder

According to the results shown in figures 13 and 14, the purity shows -7.6 % accumulated deviation from the existent model builder example, and the recovery shows -7.3% accumulated deviation.

For instance, for the one bed RPSA example gPROMS® ModelBuilder, with successive substitution, the infinity norm equals 10^{-3} after 1000 cycle simulations ($\|R(x^{1000})\|_\infty \approx 10^{-3}$). However, typically, an accuracy of 10^{-5} or better is required, meaning that, evidently further methods that display more rapid convergence should be explored. This leads to the urge to use different solvers to converge for the CSS, such as specific solvers, with the methods described in section 2, and whose results are presented in the following section 4.4.

4.4 Results with the solver

The one bed RPSA gPROMS® ModelBuilder example was accelerated with a prototype cyclic solver with the JFNK method described in section 2.10.6, a gPROMS®-based application (gBA), which is an independent software program that relies on gPROMS® engine ("server") and one or more gPROMS® Models.

4.4.1 Prototype Cyclic Solver with JFNK method

This cyclic solver is applied to the state variables of the system. It can execute sensitivity analysis and monitor key performance parameters. Being a gBA, the solver has recourse to the Event FPI implementation of the Foreign Process Interface, which provides a simple manner for the user of the Model to communicate with the dynamic simulation, allowing the use of the gPROMS® for testing event-based FPI-based communications in models that are ultimately destined to be exported for use within gBAs as well. In order to use this application, FPI := "EventFPI" is added to the SOLUTION PARAMETERS section as well as tasks to GET and SEND the variables:

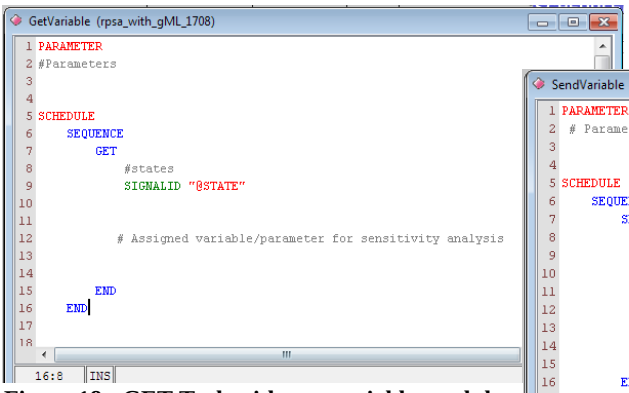

Figure 18 - GET Task with sate variables and the variables/parameters for sensitivity analysis

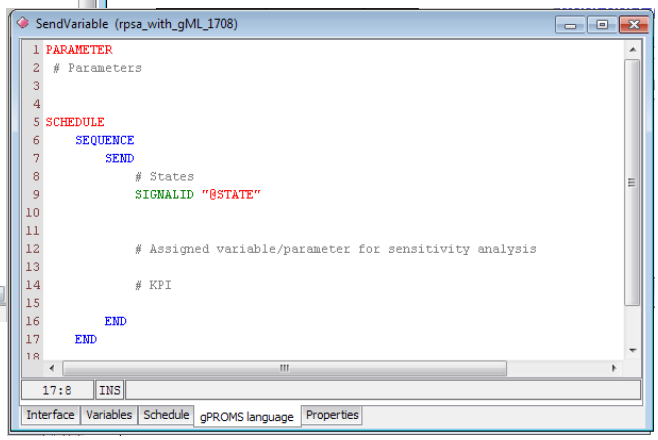

Figure 17 - SEND Task with sate variables, the variables/parameters for sensitivity analysis and the KPI's

This tasks need to be included in the file in a way that a first SEND is executed before the first cycles (including the initial state of the system). Then, a GET shall be executed before the cycle and a SEND after the cycle (with the final state of the system after that cycle). This can be done, for instance, with help of a main task highlighted in blue, part of the gML Separations –Adsorption library:

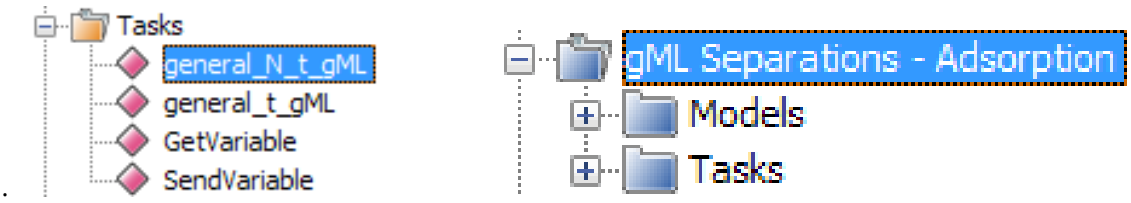

Figure 19- Process Builder tasks and models changed in this conversion

It can also be done with new tasks, added by the user, as long as they execute the operations SEND and GET at the correct points of the simulation.
A file with the initial state variables named CSS_Progress needs to be available. This file must be available each time the solver runs. However, in order for the solver to work, Restore "CSS_Progress" must be executed at the beginning (the line must be well positioned in the SCHEDULE section of the PROCESS). The initial values are restored in the system. Also, SAVE "CSS_Progress" must be present in a task and executed after each cycle. This updates the CSS_Progress file with the values of the state variables at those points.

Another file, a configuration file, needs to be available for the solver as well. This is the file that includes specifications the solver needs to access the correct input files (the exported process, with extension .gENCRYPT for instance), as well as some solving parameters including the following:

(1) Maximum number of successive substitutions prior to Newton-Krylov iteration;
(2) Stopping tolerance on f-norm;
(3) Right preconditioning method;
(4) Method for epsilon of finite-difference perturbation;
(5) Constant ϵ_r used for computing epsilon;
(6) Selection of forcing term ξ;
(7) Initial value of ξ in (0, 1) for forcing term ξ ;

Results

The first acceleration **results**, with the cyclic solver with JFNK method, for the one Bed RPSA ModelBuilder example are the following:

- Column.Purity = 0.920422
- Column.Recovery = 0.0341193

The values are equal to the ones obtained with the successive substitution in the same example, which were: 0.920421 and 0.0341193 respectively.

4.4.2 Sensitivity analysis

In this case, a study was made in order to understand the effects of the following solution parameters detailed in the configuration file of the solver:

The Maximum number of successive substitutions prior to Newton-Krylov iterations can be changed in order to give a better /more stable initial state, so that the solver calculations are not based on values so far from CSS.

Stopping tolerance of the f-norm should be studied in order to know what it takes as the required f-norm decreases, and to realize if the solver has results accurate enough at a 'good computational cost'.

The effect of right preconditioning must be studied, since from previous work, inclusively at PSE it is known to accelerate the convergence of the method. (Pattison, et al., 2014)

The method for epsilon finite-perturbation should be tested in order to know how the solver behaves accordingly to this parameter. The selection of the values of perturbation parameter ϵ is important because it must be large enough to prevent numerical noise effecting the accuracy of the derivative, but it still needs to be small enough to accurately approximate the derivative. In JFNK there is only one perturbation parameter that must be selected for all states (rather than finite differences, where an appropriate perturbation value can be found for each state). So this is crucial to properly selecting the magnitude of the perturbation parameter.

The selection of forcing term eta, the initial value of ξ and the methods used to obtain the forcing term are also very important parameters for the solver performance. Instead of considering the normal Newton correction, the Newton correction can be relaxed to an inexact Newton condition:

$$\|F(x_k) + F'(x_k)\ s_k\| \leq n_k\ \|F(x_k)\| \tag{55}$$

Where the forcing term $n_k \in [0,1]$ can be specified in various ways as described in (Eisenstat, et al., 1996) leading to a different convergence. An initial s_k satisfying the previous equation is determined using a Krylov

subspace method to solve the newton correction equation approximately that is why this method is a Newton-Krylov method. In this case, the Krylov subspace method used is GMRES. Once an initial s_k has been determined, it is tested and, if necessary reduced in length through a safeguarded backtracking until an acceptable step is obtained.

Choosing the forcing terms can lead to different results. In fact, desirably fast local convergence can be obtained by using suitably small values for the initial forcing terms near a solution. The initial forcing terms can also affect the performance of the algorithm away from a solution. – in case of considering an initial n_k too small, the Newton equation can be over solved, meaning imposing an accuracy on an approximate solution s_k of the newton equation that leads to significant disagreement between F($x_k + s_k$) and its local linear model $F(x_k) + F'(x_k)\ s_k$. This may result in little or even no progress towards a solution, involving probably a pointless expense since a less accurate solution may be both cheaper and more effective in reducing the norm of F. In the solver results, the backtracking was set off.

Regarding the performance of the solver there are **nine observable indicators**, described in section 2.6, that can be analysed while changing the parameters specified in the configuration file of the solver referred in section 4.4.1:

- Number of function evaluations
- Number of Jv - ($Jacobian \times vector$) - evaluations
- Number of P(inverse)v - ($Preconditioner^{-1} \times vector$) - evaluations
- Number of linear iterations
- Number of nonlinear iterations
- Number of backtracks
- CPU time on successive substitution
- CPU time on Newton-Krylov iteration
- Total CPU time: CPU time on successive substitutions plus CPU time on Newton-Krylov iteration.

There is also the termination flag, that being 0 means the solver did everything specified by the user, and found CSS and that is the reason for the termination of the execution. However, the termination flag can change in case the solver does the maximum iterations allowed by the user in the configuration file (meaning, with those parameters the solver is taking too much time finding the solution).

All the results of the sensitivity analysis were made with Pressure of the feed set for sensitivity analysis performed by the solver, meaning that watch results corresponds to finding 10 CSS, one for each Pressure Value between 2,1 kPa and 3.5 kPa. (See table 13).

In order to study the nine performance indicators described above, in the one bed RPSA gPROMS® ModelBuilder example, the following parameters were studied and assigned with the values mentioned in each case:

(1) Maximum number of successive substitutions prior to Newton-Krylov iteration: 10, 20, 40, 100:

A reduction in the computational time is observed as the number of successive substitutions prior to Newton-Krylov iterations increase, this was expected due to the fact that this allows a better initial point (solution), accelerating the convergence to the CSS. However, for 100 iterations, the results are not better than for 40 direct iterations, meaning is it probably not worth it to do so many successive substitutions. The results are presented for the performance of the solver parameters described above.

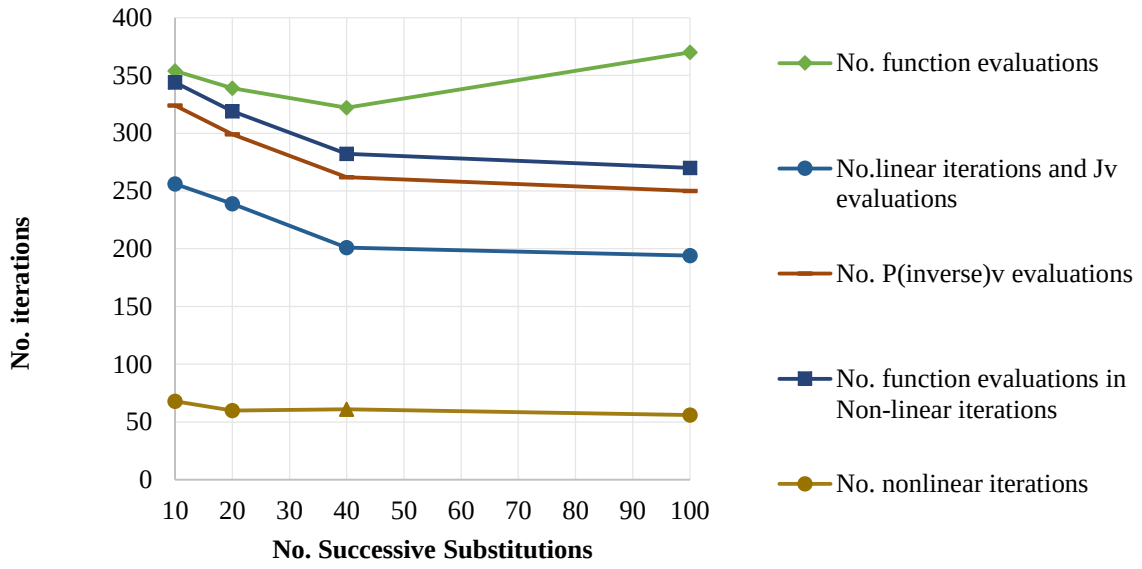

Figure 20 - Number of iterations regarding the number of Successive Substitutions prior to JFNK method

Figure 20 shows that the number of linear iterations and Jacobian product's evaluations, as well as the Preconditioner's and the function evaluations in non-linear iterations vary inversely with the number of successive substitutions.

However, with 100 successive substitutions, the results are not as better when compared with 40 substitutions, as it was expected. In fact, the number of function evaluations with 100 substitutions is much more 'expensive' computationally.

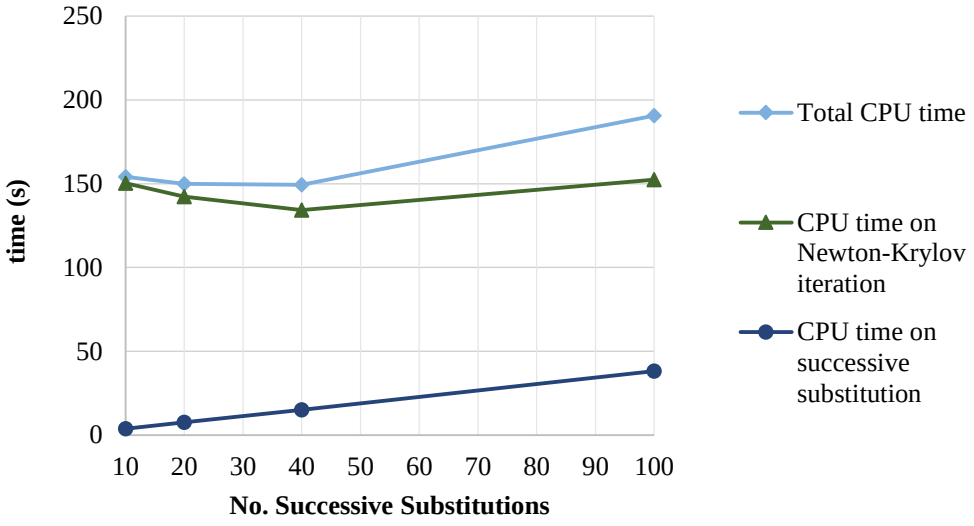

Figure 21 - Required CPU time regarding the number of Successive Substitutions prior to JFNK method

Considering 100 Successive substitutions, the total CPU time that does not follow the expected tendency registered while studying Successive substitutions between 10 and 40. A hundred successive substitutions require, in the successive substitution part of the execution of the gBA, a CPU that is too high and cannot be compensated by the CPU time on Newton-Krylov method because the initial guess obtained with this amount of successive substitutions is not that better when compared to the one obtained with 40.

A value of successive substitutions between 10 and 40 is reasonable and enough to prevent some solving errors that could lead to inaccurate solutions, by giving a sufficiently good initial guess without compromising the computational effort required to converge. Moreover, this approach can even decrease the CPU time spent in the Newton-Krylov iterations for the same stopping tolerance.

(2) Stopping tolerance on f-norm: 1e-5, 1e-6, 1e-7

The smaller the tolerance on f-norm, the longer it takes to reach the CSS. Total CPU time increases as the f-norm decreases, the solver needs to perform more Newton-Krylov iterations in order to get results with a smaller residual.

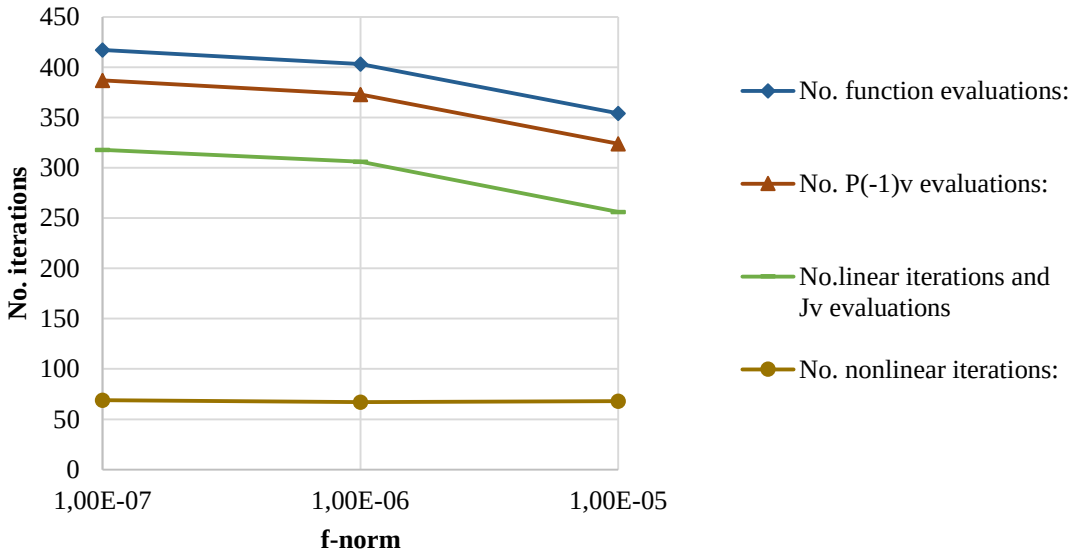

Figure 22 - Number of iteration regarding the tolerance on f-norm

This result appears as expected for iterative methods – the higher the number of iterations, the higher is the accuracy. Moreover a higher number of iterations for the same problem, with the same solving parameters requires a higher computational time, as shown in table 6.

Table 6 - Results regarding the tolerance on f-norm

CPU time	1e-5	1e-6	1e-7
CPU time on successive substitutions	3.8	3.8	3.7
CPU time on Newton-Krylov iterations	148.8	169.7	175.5
Total CPU time	152.6	173.5	179.2

(3) Right preconditioning method: 0, 1, 2

In order to evaluate the influence of the preconditioning in the solver performance, three methods were considered.

Table 7 - Precondition methods

Correspondent value of the parameter	Type of Precondition
0	No precondition
1	Precondition without updated precondition matrix during successive substitution
2	Precondition with updated precondition matrix during successive substitution

Preconditioning is the application of a transformation, the Preconditioner. The Preconditioner conditions a problem turning it more suitable for numerical solving methods.

The right preconditioning shows a faster convergence, as was expected from previous works. The right preconditioning decreases the CPU time spent on Newton-Krylov iterations, number of linear iterations and product of the Jacobian evaluations when compared to no preconditioning and 'wrong' preconditioning with updated precondition matrix during successive substitution .

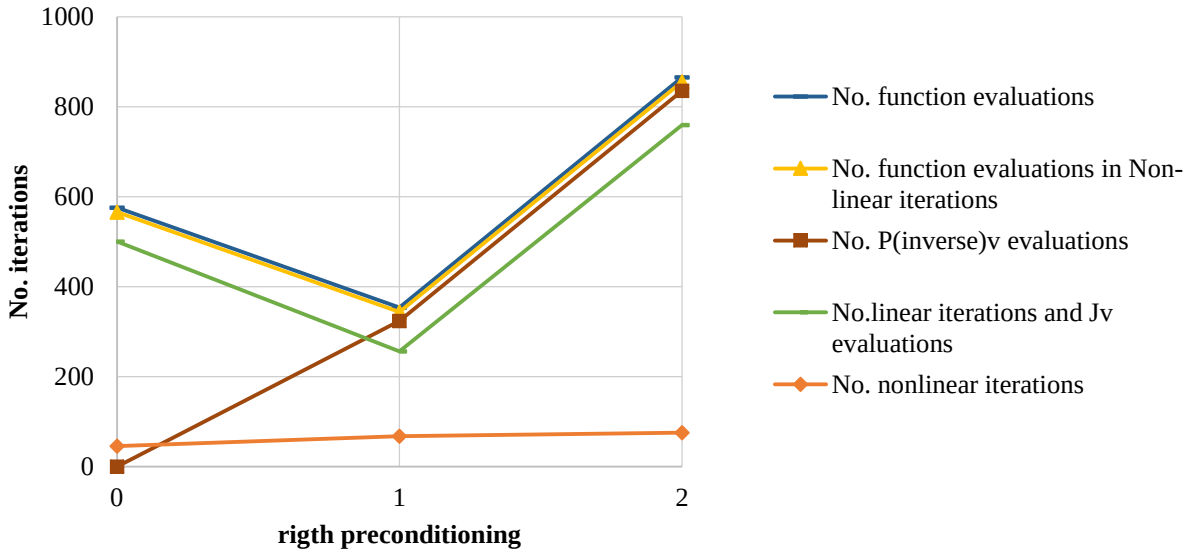

Figure 23 – Number of iterations regarding the preconditioning method

The Precondition without updated precondition matrix during successive substitution problem turned the problem more suitable for numerical solving methods, which is reflected in the decrease of all the evaluations represented in figure 23. (The solver had less 'effort' to return the solution).

Without preconditioning, the number of evaluations of the product of the Preconditioner's inverse is zero, since there is no Preconditioner. Studying the results, it is noticeable that increases from no preconditioning to preconditioning type 1 and 2 respectively. The time spent on updating the precondition matrix during successive substitution (Table 8) is very high because it leads to an increase in: the number of function evaluations in linear and nonlinear iterations, the number of the evaluations of the Preconditioner's inverse times a vector v, the number of linear iterations and number of evaluations of the Jacobian's product and the number of nonlinear iterations

Table 8 - Results regarding the Preconditioning

Right Preconditioning	**0**	**1**	**2**
CPU time on successive substitution	3.8	3.8	3.8
CPU time on Newton-Krylov iteration	241.3	148.8	364.9
Total CPU time	245.1	152.6	368.7

.

(4) Method for ϵ of finite-difference perturbation: 1, 2, 3, 4

The equations correspondent to each method are described in the table bellow.

Table 9 - Methods for ϵ of finite-difference

Method	**Calculation**
1 - "Average" epsilon	$\frac{b}{\|v\|_2 N}\sum_{i=1}^{N}\left(1+\left\|x_i^k\right\|\right)$
2 - Brown and Saad	$\frac{b}{\|v\|_2}\left\|x^{k^T}v\right\|$
3 - NITSOL	$\frac{b}{\|v\|_2}\sqrt{1+\|x^k\|_2}$
4 - This work (by suggestion of Prof. Costas Pantelides)	$b\left\|\frac{1+\left\|x_i^k\right\|}{\delta+\|v_i\|}\right\|_\infty$

Where several conditions need to be true to compute the perturbation parameter in a correct way :

- $\epsilon|v_i| > b|x_i|$. This ensures that the approximation is not affected by numerical noise.
- $\epsilon|v_i| < u|x_i|$ to ensure that it is a good approximation of the derivative for each x_i.

Therefore $b|x_i| < \epsilon|v_i| < u|x_i|$. Where b depends on the accuracy to which the equations are solved ($b = \sqrt{\epsilon_r}$), and u depends on the nonlinearity of the system (should be smaller if higher nonlinearity), Thus, $\epsilon \geq b \max \frac{|x_i|}{|v_i|}$, but only if $\frac{|v_i|}{|x_i|} > \delta$, and $\epsilon \leq u \min \frac{|x_i|}{|v_i|}$.

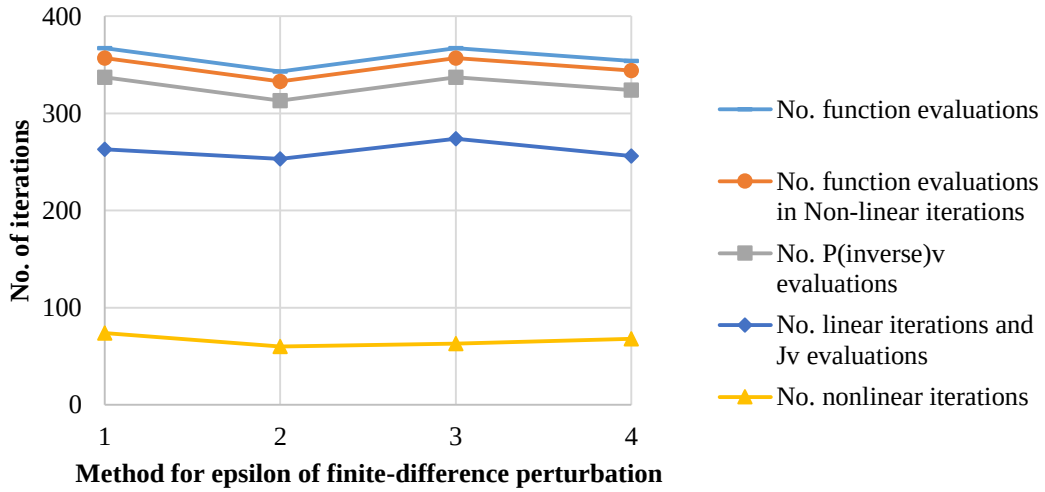

Figure 24 - Number of iterations regarding the Method for epsilon of finite-difference perturbation

For this specific example, the method that shows a slightly faster convergence is method "average" epsilon. Method 1 and 3 presented very similar results, and the same happened for method 4 and 2. In fact none of the method was especially faster, as it was expected after comparing the methods in Figure 24, since the number of iterations and evaluations show proximate values.

(5) Constant ϵ_r used for computing epsilon: 1e-5, 1e-6, 1e-7

The constant ϵ_r that shows better results is 1e-6. Probably because, as described before, the selection of perturbation parameter ϵ must be large enough to prevent numerical noise effecting the accuracy of the derivative, but it still needs to be small enough to accurately approximate the derivative.

Table 10 - Performance results regarding the constant ϵ_r used for computing epsilon

ϵ_r	1E-05	1E-06	1E-07
Termination flag iterm	0	0	0
No. function evaluations	912	338	354
No. J*v evaluations:	590	243	256
No. P(inverse)v evaluations	804	308	324
No. linear iterations	590	243	256
No. nonlinear iterations	214	65	68
No. backtracks	0	0	0
CPU time on successive substitution	3.8	3.8	3.8
CPU time on Newton-Krylov iteration	465.2	140.9	148.8
Total CPU time	469.0	144.6	152.6

The smaller ϵ_r, the smaller will be b, as well as the perturbation parameter, taking into account the equations in table 9. This lack of accuracy in the derivative leads to a higher number of both linear and nonlinear iterations, which is reflected in more Jacobian's product evaluations.

(6) Selection of forcing term ξ : 0, 1, 2, 3

Regarding the method to obtain the forcing term eta, the faster is 0. However the results do not change a lot when this parameter is changed.

Table 11 - Results regarding forcing term

Forcing term	0	1	2	3
Termination flag iterm	0	0	0	0
No. function evaluations	337	354	354	383
No. J*v evaluations:	241	256	256	302
No. P(inverse)v evaluations	307	324	324	353
No. linear iterations	241	256	256	302
No. nonlinear iterations	66	68	68	51
No. backtracks	0	0	0	0
CPU time on successive substitution	3.8	3.8	3.9	3.8
CPU time on Newton-Krylov iteration	143.8	148.8	150.4	162.1
Total CPU time	147.6	152.6	154.3	165.9

This analysis is redundant when compared with the following analysis.

(7) Initial value of ξ in (0, 1) for forcing term ξ in method 1, 2 and 3: 0.5, 0.7, 0.9

For the last parameter, the best results are for method 1, showing a different behaviour depending on the initial value of eta. In fact, regardless the method for forcing term eta, some parameters do not change when the initial value of eta is the same for two methods (Number of function evaluations with initial value 0.5 in method 1 and 2).

The number of nonlinear iterations is the same in method 1 and 2 for each initial value, as well as the number of Jacobian's product evaluations, number of Preconditioner's inverse product evaluations, and number of linear iterations. The only differences are in the CPU time.

Table 12 - Results regarding the Initial value of ξ in (0, 1)

Method	**1**			**2**			**3**		
term eta	0.5	0.7	0.9	0.5	0.7	0.9	0.5	0.7	0.9
Termination flag iterm	0	0	0	0	0	0	0	0	0
No. function evaluations	354	388	358	354	388	358	383	383	383
No. J*v evaluations:	251	276	243	256	276	243	302	302	302
No. P(inverse)v evaluations	324	358	328	324	358	328	353	353	353
No. linear iterations	256	276	243	256	276	243	302	302	302
No. nonlinear iterations	68	82	85	68	82	85	51	51	51
No. backtracks	0	0	0	0	0	0	0	0	0
CPU time on successive substitution	3.7	3.8	3.9	3.9	3.7	3.8	3.8	3.9	3.9
CPU time on Newton-Krylov iteration	148.4	164.8	152.7	150.4	164.3	150.1	162.1	163.3	163.4
Total CPU time	152.2	168.5	156.6	154.3	168.0	153.9	165.9	167.2	167.3

Desirably a fast local convergence can be obtained by using suitably small values for the initial forcing terms near a solution, which should be the case since 10 successive substitutions are a good initial guess for the solver. In fact, the faster values hold for the smaller initial value (0.5), meaning that this choice probably did not lead to a pointless expense on contrary as what was explained could happen in the beginning of this section.

4.4.3 Mixed Sensitivity analysis

In order to do a mixed sensitivity analysis, parameters were changed randomly for different number of successive substitutions. This analysis was performed for the one bed RPSA ModelBuilder example. The faster result was for 20 successive substitutions with the following values:

Table 13 - Specified parameters for the fastest study in mixed sensitivity analysis.

Description	Value
gPROMS® output level	9
gCSS output level	4
number of KPIs to be monitored	2
number of parameters for sensitivity analysis	1
target values of parameters for sensitivty analysis	350 000
number of sampling points	10
maximum number of successive substitutions	**20**
maximum number of Newton-Krylov iterations	2 000
function norm tolerance	1e-6
step length tolerance	1e-10
maximum Krylov subspace dimension	2 000
right preconditioning	**1**
maximum number of iterations per Krylov	1 000
residual update on restart	0
method for epsilon of finite difference	**2**
constant emr used for computing épsilon	1e-6
maximum number of backtracks	-1
forcing term	**1**
initial eta	**0.5**
alpha and gamma	2 1
fixed eta	0.1

After studying the 20 results, in the majority of the cases, method 2 for finite difference is the faster while method 4 is the slower.

The fastest examples of this mixed sensitivity analysis all hold for Preconditioning method 1 either for 20, 40 or 100 successive substitutions. The constant ϵ_r, used for computing epsilon, in the fastest solver performance is 10^{-6}, the same as concluded in section 4.4.2 (5), and the initial eta obtained in the study with the parameters

above was the fastest obtained, for this parameter in section 4.4.2 (7). Concerning the method used for computing epsilon, method 4 and 2 showed the best results. However, for the forcing term method, this study showed the fastest results for method 2, which was the second fastest in section 4.2.2 (6). From this analysis, method 1 and 2 are the best for this problem.

After this sensitivity analysis, the majority of the solver parameters combinations lead to the CSS, and besides the fact that some took hours instead of minutes, all of them were accurate when compared to the ones obtained for the KPI's (Purity and Recovery) during successive substitution, since all of them converged to the same CSS.

This page was intentionally left blank.

5. Conclusions

During the development of this project, several aspects of modelling and simulation in general using the gPROMS language were studied. In particular, simulating bed models is a difficult task due to all the Partial Differential Algebraic Equations that described these models which need a specific treatment while being solved. In fact, after converting a Rapid Pressure Swing Adsorption (RPSA) example from the gPROMS® ModelBuilder into Process Builder obtained results were not exactly the same. This fact enhances the idea that the results of the simulation of this type of processes highly depend on the solving method and the models used. For the RPSA example the conversion from gPROMS® ModelBuilder into ProcessBuilder displays a deviation of approximately 7% using two key performance indicators as purity and recovery ratios.

Due to the periodical behavior of these systems, the convergence to the Cyclic Steady State is required in order to simulate and optimize this process. This characteristic makes these problems even more difficult to handle. Considering successive substitution for the convergence of this type of processes is incompatible with the user expectations since becomes too time consuming. However, for a one bed Rapid Pressure Swing Adsorption gPROMS® ProcessBuilder example using the Wesgtein method this convergence can be accelerated showing more promising results in terms of computational time without putting in danger the accuracy of the solution. The residual (the highest registered difference of two consecutive iterations between all the considered state variables – infinity norm) obtained was 5.32×10^{-6} after 6 iterations with the Wegstein method (smaller than the result with successive substitution: 2.69×10^{-4}).

Considering the usage of the cyclic solver with Jacobian-Free Newton Krylov (JFNK) method, the ModelBuilder example displays much faster convergence to the Cyclic Steady State with JFNK than successive substitution. The Cyclic Solver's results for the Cyclic Steady State are 'accurate' since the Key Performance Indicators obtained are the same as with successive substitution. After performing the sensitivity analysis, the following conclusions were taken: A good initial guess for the solver is easily found with 10 to 40 Successive Subsitutions prior to JFNK method. Moreover, in this case, Precondition without updated precondition matrix during successive substitution shows even faster results when compared to no Precondition and Precondition with updated precondition matrix during successive substitution. The other parameters, such as the method for finite differences do not influence the performance of the solver as much.

Taking into account the one bed RPSA Process Builder example validated in this project, it could be concluded that this example, nor the solver are yet prepared to be used together. The solver requires specific operations in the gPROMS® models that are not easily accomplished by the user. Moreover, the solver application leads to errors that are not easy to be detected or solved.

Future works would be to investigate optimisation for dynamic problems such as Pressure Swing Adsorption and other periodic processes with simple examples using gPROMS®. In order to do so, improving the solver configuration file, and output layout (errors descriptions), turning it more convenient and easy to use is fundamental.

This page was intentionally left blank.

6. Bibliography

A parametric study of layered bed PSA for hydrogen purification. **Ribeiro, Ana M., et al. 2008.** 2008, Chemical Engineering Science 63, pp. 5258–5273.

An Introduction to the numerical method of lines Integration of Partial Differential Equations. **Schiesser, E. W. 1977.** s.l. : Lehigh universities and Naval Air Development Center, 1977, Differential Sytems Simulator, Version 2.

Aspen Technology, Inc. 1998. Aspen Plus User Guide. s.l. : Aspen Technology, Inc, 1998. Vol. 2.

Barton, P.I and Pantelides, C.C. 1994. Modeling of Combined Discrete/Continuous Processes. *AIChE Journal, 40(6):.* June 1994, pp. 966–979.

Bastos-Neto, et al. 2011. Breakthrough curves of methane at high pressures for H2 purification processes. *Chemie Ingenieur Technik, 83(1-2).* 2011, pp. 183-190.

Batta, L.B., Island, G.,. 1971. *Selective adsorption process. US Patent 3564816* 1971.

Cavenati, S, Grande, CA and Rodrigues, AE. 2005. Upgrade of methane from landfill gas by pressure swing adsorption. *Energ Fuel.* 6 19, 2005, pp. 2545-5.

Chahbani, M.H. and D. Tondeur,. 2010. Predicting the final pressure in the equalization step of PSA cycles. *Separation and Purification Technology 71.* 2010, pp. 225-232.

Chlendi, M and Tondeur, D. 1995. *Dynamic behaviour of layered columns in pressure swing adsorption. Gas Separation & Purification.* 1995. pp. 231-242.

Conney, David O. 1998. *Adsorption Design for Wastewater Treatment.* London : CRC Press, 1998.

Cruz, P and al, et. 2003. *Cyclic adsorption separation processes: analysis strategy and optimization procedure.* s.l. : Chemical Engineering Science, 2003.

Cruz, P, Magalhães, F. D and Mendes, A. 2005. On the Optimization of Cyclic Adsorption Separation Processes. *AIChE Journal.* 2005, Vol. 51 (5).

Ding, Yuqing, Croft, T David and LeVan, M Douglas. 2002. Periodic states of adsorption cycles IV. Direct optimization. *Chemical Engineering Science 5.* 2002, Vols. 57(21):4521-4531.

Eisenstat, S. C and Walker, H. F. 1996. Choosing the forcing terms in an inexact newton method. *SIAM J: Sci.Comput.* 1996, Vol. 17.

Equilibrium Theory analysis of dual reflux PSA for separation of a binary mixture. **Ritter, A. D. Ebner and J.A. 2004.** 2004, AIChE Journal, vol 50, no 10, pp. 2418-2429.

Equilibrium theory analysis of rectifying PSA for heavy component production. **Ebner, A. D and Ritter, J. A. 2002.** 2002, AIChE Journal 48(8), pp. 1679-1691.

Equilibrium theory for solvent vapor recovery by pressure swing adsorption: analytical solution for process performance. **Subramanian, D.,Ritter,J.A. 1997.** 1997, Che-mical Engineering Science 52(18), pp. 3147-3160.

Finlayson, Bruce A. 2014. *Introduction to Chemical Engineering Computing.* New Jersey : John Wiley & Sons, 2014.

Gadkaree, K. P. 1998. Carbon honeycomb structures for adsorption applications. *Carbon Vol.36 No 7-8.* New York : Corning Inc., 1998, pp. 981-989.

García S. et al, S. 2013. Cyclic operation of a fixed-bed pressure and temperature swing process for CO2 capture: Experimental and statistical analysis 12(0). *International Journal of Greenhouse Gas Control.* 2013, pp. 35-43.

Gittleman C. et al. 2005. *Hydrogen purification process using pressure swing adsorption for fuel cell applications.* Google Patents, 2005.

Grande Carlos Adolfo, Carlos Adolfo and Cavenati, Simone Rodrigues A. 2008. *WO2008072215 A2* Porto, 2008.

Grande, C. A. 2012. *Advances in pressure swing adsorption for gas separation.* s.l. : International Scholarity Research Notices 2012, 2012.

Heinrich, K. 1953. *Process for the purification and separation of gas mixtures.* Googgle Patents, 1953.

Jee, J. G, Kim, M.B and Lee, C. H. 2001. *Adsorption Characteristics of Hydrogen Mixtures in a Layered Bed: Binary, Ternary, and Five-Component Mixtures.* s.l. : Industrial & Engineering Chemistry Research, 2001.

Jiang, L, Biegler, L.T and Fox, V. G. 2003 . Simulation and Optimization of Pressure e Swing Adsorption System for Air Separation. *AIChE Journal.* 2003 , Vol. 49 (5).

Jiang, L., Fox, V.G and Biegler, L.T. 2004. Simulation and optimal design of multiple-bed pressure swing adsorption systems. *AIChE J., 50(11).* 2004, pp. 2904-2917.

Joankin, Beck and S. Fraga , Eric. 2012. Workshop on Mathematical Modelling and Simulation of Power Plants and CO2 Capture. *Surrogate Modelling for PSA Design for.* s.l. : UCL, 2012.

Knaebel, Kent, S. 2004. Adsorbent Selection. Dublin : Adsorption Research Inc., 2004, p. http://www.adsorption.com/publications/ AdsorbentSel1B.pdf.

Knoll, D.A and Keyes, D.E. 2004. Jacobian-free Newton–Krylov methods: a survey of approaches and applications. *Journal of Computational Physics.* 2004, Vol. 193(2).

Ko, D and Moon, I. 2002. Multiobjective optimization of cyclic adsorption processes. *Ind. Eng. Chem. Res.* 2002.

Ko, D., Siriwardane, R and Biegler, L.T. 2003. Optimization of a pressure-swing adsorption process using zeolite 13X for CO2 sequestration. *IECR, 42(2).* 2003, pp. 339-348.

—. 2005. Optimization of pressure swing adsorption and fractionated vacuum pressure swing adsorption processes for CO2 capture. *IECR, 44(21).* 2005, pp. 8084-8094.

Le Lann, J.M., et al. 1998. Dynamic simulation of Partial Differential Algebraic Systems. *Application to some Chemical Engineering problems.* 1998.

Lee, Chang Ha. 2003. *Adsorption science and technology: Procedures of the third pacific basin conference.* s.l. : World Scientific publishing Co, Pte ltd, 2003.

Liu, Z., et al.,. 2011. Multi-bed Vaccuum Pressure Swing Adsorption for carbon dioxide capture from flue gas. *Separation and Purification Technology.* 2011, pp. 307-317.

Long, D, Earls, E and G.N. 1980. *Multiple bed rapid pressure swing adsorption for oxygen, US patent 4, 194, 891.* 1980.

Luyben, W.L. 1992. *Practical Distillation Control.* New York : Springer Science & Business Medi, 1992.

Malek, A. and S. Farooq,. 1997. Study of a six-bed pressure swing adsorption process 43(10). *AIChE journal.* 1997, pp. 2509-2523.

Mayers, A. L. 2005. Prediction of Adsorption of Nonideal Mixtures in Nanoporous Materials. *Adsorption 11.* 2005, pp. 37 -42.

New PSA process with intermediate feed inlet position operated with dual refluxes - application to carbon-dioxide removal and enrichment. **Diagne, D, Goto, M and Hirose, T. 1994.** 1994, Journal of Chemical Engineering of Japan, pp. 85-89.

Nilchan, S. and Pantelides, C.C. 1998. On the Optimisation of Periodic Adsorption Processes. *Adsorption.* 1998, Vol. 4, pp. 113-147.

Oliver J. Smith, Arthur W. Westerberg. 1992. *Acceleration of cyclic steady state convergence for.* Pittsburgh : Ind. Eng. Chem. Res, 1992.

Polanyi, M. 1932. Section III - theories of the adsorption of gases. *Transactions of the Faraday Society.* 1932, pp. 316-333.

Python. 2016. Python. *Python.* [Online] May 25, 2016. https://www.python.org/.

Rao, G. Spoorthi · R.S. Thakur · Nitin Kaistha · D.P. 2010. *Process intensification in PSA processes for upgrading synthetic.* s.l. : Springer Science+Business Media, 2010.

Renou, Elise, Monereau, Christian and Carriere, Celine. 2015. *Psa process with one active step per phase time. US 20150143993 A1* May 28, 2015.

Richardson, J. F., Harker, J. H. and Backhurst, J. R. 2002. *Chemical Engineering (5th Edition) Volume 2: Particle Technology and Separation Processes 002.* 2002. pp. 970-1052.

Rota, R. 2015. *Separation process of gaseous compounds from natural gas with low exergy losses, Patent EP 2958655 A1.* 2015.

Ruthven, D.M, Farooq, S. and Knaebel, K.S. 1994. *Pressure swing adsorption.* s.l. : VCH Publishers, 1994.

Ruthven, D.M. 1984. *Principles of Adorption and Adsorption Processes.* New York : John Wiley & Sons, 1984.

Schiesser, E. W. 1991. *The Numerical Method of lines.* New York : Academic Press, 1991.

Serbezov, A. 2001. Effect of the process parameters on the length of the mass transfer zone during product withdrawal in pressure swing adsorption cycles. *Chemical Engineering Science, vol 56, no 15.* 2001, pp. 4673-4684.

Sircar, S. 2006. Basic Research needs for the Design of Adsorptive Gas SeparationProcesses. *Ind. Eng. Chem. Res., 45 (16).* 2006, pp. 5435–5448.

Sircar, S. and Golden, T.C. 2000. *Purification of Hydrogen by Pressure Swing Adsorption. Separation Science and Technology.* 2000.

Skarstrom, C. W. 1959. Use of adsorption phenomena in automatic plant – type gas analysers. *Annuals of the New York Academy of Sciences.* 1959, pp. 751-763.

Staudt, J. Keller and R. 2005. *Gas Adsorption Equilibria: Experimental Methods and Adsorption Isotherms.* Boston : Springer, 2005.

Y. Lu, S. Doong, M.Bulow, et al. 2004. Pressure-Swing Adsorption Using Layered Adsorbent Beds with Different Adsorption Properties II - Experimental Investigation. *Adsorption 10.* 2004, pp. 267-275.

Yamaguchi, T., Kobayashi, Y. 1993. *Gas separation process. US Patent 5250088* 1993.

Yang, R. T. 2003. *Adsorbents. Fundamentals and Applications.* New York : John Wiley & Sons, 2003.

7. Appendix

7.1 Python syntax

To run gPROMS® only one command line is needed. But, first it is required to set the correct path.

The command line needed includes the gORUN arguments and has the following form:

subprocess.check_call(["**gORUN.bat", "name_of_encrypted_file.gENCRYPT", "type_of_opperation", "name_of_the_process", "password"**])

Some useful functions already defined in python are useful to do this program:

- Writing a file
  - *fr*=open*(***"Name_of_the_file.txt"**, **"w"***)*
    fr.write(text)
    fr.close()

In this function "text" should be a string containing what you want to write in the file.

- Reading a file
  - fw=open(**"Name_of_the_file_to_copy"**, **"r"**)
    text=fw.read()
    fw.close()

This function saves in the variable 'text' the string equal to the text of the whole file.

- Removing a file
  - os.remove(**"Whole_directory_to_the_file"**)
- Save a line of a text file in a variable
  - f = open(**"name_of_the_file"**)
    line=linecache.getline(**'name_of_the_file'**, line_number)
    f.close()
    text = eval(**"line"**)

7.2 Python Program

Table 14 - Python Program

1 Calls goRUN	2 Starts simulation: 2.1-Writes send-text-file1 with initial values 2.2-Aborts simulation because get-text-file-1 is not available
3 Does operations with text files and calls gPROMS®: 3.1-Reads send-text-file-1 and saves the values in a vector x1 3.2-Writes get-text-file-1 with the initial values in the send-text-file-1 3.3-deletes send-text-file-1 3.4-Calls gPROMS®	3 Starts simulation: 4.1- Writes send-text-file-1 4.2-Reads get-text-file-1 4.3-Does one iteration/cycle simulation 4.4- Writes send-text-file-2 with final values after the first iteration/cycle
5 Does operations with text files and calls gPROMS®: 5.1-Reads send-text-file-2 and saves the values in a vector x2 5.2-Writes get-text-file-1 with the final values of the first cycle present in the send-text-file-2 in order to let gPROMS® use the final values of the first iteration/cycle as the initial values of the second iteration/cycle. 5.3-deletes send-text-file-1 and send-text-file-2 5.4-Calls gPROMS®	6 Starts simulation: 6.1- Writes send-text-file-1 6.2-Reads get-text-file-1 6.3-Does one iteration/cycle simulation 6.4- Writes send-text-file-2 with final values after the second iteration/cycle
7 does operations with the text files, applies the Wegstein method and (calls gPROMS®):	8 Starts simulation:

7.1-Reads send-text-file-2 and saves the values in a vector x3 7.2 – Applies the acceleration and saves the solution in a vector w1 and the residual of all variables in a vector r1 7.3 – saves the f-norm in a increasable vector in position n 7.4-Writes get-text-file-1 with the values of the updated Wegstein solution present in the vector w1 in order to let gPROMS® use the accelerated solution as the initial values of the next iteration/cycle. 7.5- deletes send-text-file-1 and send-text-file-2 and assigns n=n+1 7.6- If the user specified in n>1 then, Calls gPROMS®	8.1- Writes send-text-file-1 8.2-Reads get-text-file-1 with the Wesgtein updated solution 8.3-Does one iteration/cycle simulation 8.4- Writes send-text-file-2 with final values after the first iteration/cycle
9 does operations with text files and calls gPROMS®: 9.1 – Reads s end-text-file-1 and saves the values in a vector x1 9.2-Reads send-text-file-2 and saves the values in a vector x2 9.3-Writes get-text-file-1 with the final values of the first cycle present in the send-text-file-2 in order to let gPROMS® use the final values of the first iteration/cycle as the initial values of the second iteration/cycle. 9.4-deletes send-text-file-1 and send-text-file-2 9.5- Calls gPROMS®	10 Starts simulation: 10.1- Writes send-text-file-1 10.2-Reads get-text-file-1 10.3-Does one iteration/cycle simulation 10.4- Writes send-text-file-2 with final values after the second iteration/cycle

11 does operations with the text files, applies the Wegstein method and (calls gPROMS®): 11.1-Reads send-text-file-2 and saves the values in a vector x3 11.2 – Applies the acceleration and saves the solution in a vector w1 and the residual of all variables in a vector r1 11.3 – saves the f-norm in a increasable vector f in position n 11.4-Writes get-text-file-1 with the values of the updated Wegstein solution present in the vector w1 in order to let gPROMS® use the accelerated solution as the initial values of the next iteration/cycle. 11.5- deletes send-text-file-1 and send-text-file-2 and assigns n=n+1 11.6- Calls gPROMS® Until n= value specified by the user	
8, 9, 10 and 11 are executed in this exact order until 11.6 condition is not true. The program returns f.	

7.3 Specifications for the model

Table 15 - Values set to the model parameters

Name	Value	Units
Number of components	2	-
Bed length	1	m
Bed density	800	Kg m^{-3}
Bed Area	1.96×10^{-3}	m^2
Bed Void	0.35	-
Particle Void	0.55	-
Total Bed Void	$\varepsilon_{tot} = \varepsilon_{bed} + \varepsilon_p (1 - \varepsilon_b)$	-
Particle diameter	302.5×10^{-6}	m
Particle radius	$\frac{Particle\ diameter}{2}$	m
Pore diameter	0.12×10^{-6}	m
tortuosity	3	-
Viscosity	1.8×10^{-5}	N s m^{-2}
Diffusivity	1×10^{-3}	m^2 s^{-1}
m [N_2 ; O_2]	3.08×10^{-6} ; 1.43×10^{-6}	mol kg^{-1} Pa^{-1}
MW[N_2 ; O_2]	28 ; 32	kg $kmol^{-1}$
R	8.314	J mol^{-1} K^{-1}
Feed Pressure	2.12×10^{-5}	Pa
Feed Temperature	290	K
Feed composition[N_2 ; O_2]	0.79 ; 0.21	-
Waste Pressure	1×10^{-5}	Pa
Atmosferic Pressure	1×10^{-5}	Pa
Volumetric Flow	1×10^{-5}	m^3 s^{-1}
Discretization method	[OCFEM,3,50]	Axial

The following specifications in the Source material reversible where given in the dialogue box of this unit:

Table 16 - Specifications of the Feed stream

Specification	Value	Units
Pressure	212000	Pa
Temperature	290	K
Molar fraction (N_2)	0.79	
Molar fraction (O_2)	0.21	
Mode	Pressure-driven (specified Pressure)	
Mass and energy accumulation	ON	
External reset	ON	
Phase for physical properties	Multiphase	

The following specifications in the Product where given in the dialogue box of this unit:

Table 17 - Specifications of the Product stream

Specification	Value	Units
Temperature	290	K
Molar fraction (N_2)	0	
Molar fraction (O_2)	1	
Mode	Pressure-driven (specified flow)	
Mass and energy accumulation	ON	
External reset	ON	
Phase for physical properties	Multiphase	

7.4 Application of Wegstein method in simulation

In order to use this algorithm, some of the following Wegstein Acceleration Parameters can be specified in order to help the convergence for each specific case:

• Acceleration parameter q Bounds
• Number of direct substitution iterations before the first acceleration
• Number of direct substitution iterations between acceleration iterations
• Number of consecutive acceleration iterations

When solving sets of nonlinear equations, it is often desirable to ensure that none of the equations converge at rates outside a pre-specified range. And so, resetting values of qi that fall outside the desired limits (i.e., $qmin < qi < qmax$) ensures this. For most flowsheets, the default lower and upper bounds on q can be -5 and 0, respectively. Normally an Upper Bound of the Wegstein acceleration parameter of 0 should be used. However, if iterations move the variables slowly toward convergence, smaller values of the lower bound (like -25 or -50) may give better results. In case of oscillating direct substitution, values of the lower and upper bounds between 0 and 1 may help. (Aspen Technology, 1998)

In PSA systems the iterate variables are not uncoupled (independent) nor only weakly coupled, and so this method is not the most correct to use. Moreover, in the program created, q is unbounded, which can lead to even more unrealistic results.

7.5 Normal procedure to convert Model Builder examples into Process Builder

When converting a model builder example into Process builder the following aspects should be taken into account:

1- Main equations in the model lead to specific equipment:

From the equations in the model, the user can decide which libraries to use. With most matter given to the main equipment, such as distillation columns, adsorption beds, reactors, etc. Since dome of the equations are not directly linked to the equipment but to the operation mode for instance, all of the equations should be checked out of a list, one by one as the topology of the Process Builder example grows. This is the only way to guarantee that all the specifications are reproduced.

Choosing the equipment should be easier if the model builder example already includes topology. (However the user must check the specifications given in the topology very carefully).

2 – Operational Conditions

Some of these can be deducted from the equations. Such as isothermal condition, Pressure drop equation, etc.

In case of containing topology, some of these can be obtained checking the topology tab. (Most of these should be easily reproduced when using Process Builder Libraries). However, the models in the topology are not exactly the same, so, special attention should be given to this part.

3 – Valves

The valves are sometimes not included in model builder examples that do not contain topology. Meaning that the user needs to know which operations are typically managed by valves, such as changing flows, pressure of the streams and so on.

4- Units and names of variables

Sometimes the units for similar libraries are not the same in Model Builder and Process Builder. It is up to the user to be especially careful with this matter.

In some of the cases it is even more difficult to reproduce the example in the other platform due to the fact that the names of the variables change sometimes to similar names that are often difficult to realize if the user is not used to chemical engineering vocabulary and does not have a clear definition of each variable and its typical units.

5- Other equations and operational conditions

Some of the equations and operational conditions are not easily specified in process Builder topology tab and require for instance the usage of models force a certain variable to have a certain behaviour /value by adjusting another variable. An example is a constant volumetric flow rate adjusting the mass flow rate.